

Branch Cut - Trabalho Final - Programação Inteira

Bruno Maciel Machado

Universidade Federal de Santa Catarina, Florianópolis, Santa Catarina, Brazil



Contents

1	Introdução	5
2	Formulação do Problema de Programação Inteira Mista	6
2.1	Relaxação linear	7
2.2	Branch-and-Bound e Branch-and-Cut	7
3	Arquitetura do solver	8
3.1	Métodos auxiliares	9
3.2	Loop principal: <code>solve()</code>	9
4	Presolve	10
4.1	Arquitetura do módulo	10
4.2	Bound Tightening	10
4.3	Row Feasibility and Redundancy	11
4.4	Substitute Fixed Variables	12
4.5	Singleton Row Reduction	12
4.6	Implicit Free Variable Detection	13
4.7	Euclidean Reduction	13
4.8	Coefficient Strengthening	13
4.9	Chvátal-Gomory Strengthening	14
4.10	Parallel Rows	14
4.11	Orquestração pelo Presolver	15
4.12	Síntese	15
5	Busca em árvore	15
5.1	Estrutura de um nó da árvore	16
5.2	Abstração <code>NodeSelection</code>	17
5.3	Estratégia Depth-First Search	17
5.4	Estratégia Breadth-First Search	17
5.5	Estratégia Best-First Search	18
6	Seleção de Variável para Branching	18
6.1	Abstração <code>BranchVariableRule</code>	19
6.2	Regra <code>FirstFractionalRule</code>	19
6.3	Regra <code>MostFractionalRule</code>	20
6.4	Regra <code>StrongBranchingRule</code>	20
6.5	Regra <code>PseudoCostRule</code>	22

6.6	Atualização dos pseudocustos	23
6.7	Integração com o solver	24
7	Geração de cortes	24
7.1	Arquitetura geral	25
7.2	Estrutura auxiliar <code>_CutRow</code> e extração do tableau	25
7.3	Reconstrução do corte no espaço original	25
7.4	Mixed-Integer Gomory Cut	26
7.5	MIR	26
7.6	Lifting & Extending	27
7.7	Cover Cut	27
7.8	Integração com o solver	27
8	Heurísticas primais	27
8.1	Rounding Heuristic	28
8.2	Família <code>DivingHeuristic</code>	28
8.3	Feasibility Pump	29
8.4	Feasibility Jump	29
8.5	Fix and Propagate	29
8.6	RENS	30
8.7	RINS	30
8.8	Local Search	30
8.9	Local Branching	30
8.10	LNS	31
8.11	ALNS	31
9	Linearização por partes via SOS2	31
9.1	Estrutura da aproximação	31
9.2	Implementação via variáveis binárias	32
9.3	Integração com o solver	33
10	Experimentos	33
10.1	Objetivos experimentais	33
10.2	Ambiente experimental	34
10.3	Configuração-base e metodologia de comparação	34
10.4	Instâncias testadas	35
10.5	Métricas coletadas	36
10.6	Validação básica	37

10.7	Avaliação do presolve	38
10.8	Comparação entre estratégias de exploração da árvore	41
10.9	Comparação entre regras de branching	46
10.10	Avaliação dos cortes	49
10.11	Avaliação das heurísticas primais	52
10.12	Comparação entre a configuração <i>baseline</i> e a configuração completa	56
10.12.1	Resultados comparativos	57
10.12.2	Qualidade da relaxação e efeito dos cortes	60
10.12.3	Redução do esforço da árvore	61
10.12.4	Impacto nos tempos de execução	62
10.12.5	Síntese da comparação	63
10.13	Teste com formulação SOS2	63
10.13.1	Estrutura da instância gerada	64
10.13.2	Resultado	64
11	Discussão	65
12	Conclusão	66

1. Introdução

A Programação Inteira Mista é uma das principais ferramentas de modelagem para problemas de otimização combinatória e decisão, sendo amplamente utilizada em aplicações como planejamento da produção, logística, escalonamento, alocação de recursos e problemas de rede. Apesar de sua expressividade, a resolução exata de problemas de Programação Inteira Mista é, em geral, computacionalmente desafiadora, especialmente quando o número de variáveis inteiras e restrições cresce. Nesse contexto, métodos baseados em relaxações lineares e enumeração implícita tornam-se fundamentais para obter soluções ótimas de forma sistemática.

O objetivo deste trabalho é implementar um solver básico de Branch-and-Cut para problemas de Programação Inteira Mista (MILP) (Seção 2). Buscou-se combinar diferentes técnicas de *presolve*, exploração da árvore de busca, estratégias de seleção de variáveis para branching, geração de cortes válidos, heurísticas primais e suporte a formulações com estrutura SOS2 para a solução de problemas do tipo MILP.

Um solver Branch-and-Cut pode ser entendido como uma extensão do método de Branch-and-Bound aliada à geração de cortes. De forma geral, o método resolve inicialmente a relaxação linear do problema, isto é, a versão em que as restrições de integralidade são temporariamente relaxadas. Caso a solução obtida já seja inteira (nas variáveis que possuem originalmente essa restrição), ela é candidata à solução ótima, o que podemos vir a chamar de solução incumbente. Caso contrário, o algoritmo pode seguir dois caminhos distintos: dividir o problema em subproblemas menores por meio de branching, criando uma árvore de busca, e/ou reforçar a formulação por meio da adição de cortes válidos, que eliminam soluções fracionárias sem remover soluções inteiras viáveis. Essa combinação tende a reduzir o tamanho efetivo da árvore, tornando a busca mais eficiente.

Neste trabalho, a resolução da relaxação linear é delegada a um *solver* externo de programação linear, especificamente o Gurobi, enquanto a lógica de Branch-and-Cut é implementada diretamente. Desta forma, os seguintes componentes foram implementados. Primeiramente, foi desenvolvido um módulo de *presolve* (Seção 4) responsável por simplificações iniciais do modelo. Em seguida, foi implementada a busca em árvore com suporte a diferentes estratégias de exploração (Seção 5), em particular Depth-First Search e Best-First Search. Para a seleção da variável de branching (Seção 6), foram consideradas diferentes regras, com destaque para Strong Branching e Pseudocost

Branching. Também foi incorporado um módulo de geração de cortes (Seção 7), incluindo cortes do tipo Gomory Mixed-Integer, Mixed-Integer Rounding e Cover Cuts. Além disso, o solver inclui heurísticas primais para obtenção de soluções viáveis (Seção 8). Por fim, foi adicionado suporte a restrições do tipo SOS2 (Seção 9) por meio de uma reformulação MILP equivalente com variáveis λ e variáveis binárias auxiliares, permitindo tratar problemas de linearização piecewise dentro da arquitetura já existente do solver. A arquitetura geral empregada no solver é descrita na Seção 3.

Na Seção 10, descrevem-se os experimentos, sua metodologia e os resultados através do impacto prático de cada componente implementado.

Dessa forma, este trabalho pretende, além de apresentar uma implementação funcional de um solver Branch-and-Cut, discutir o impacto e como diferentes componentes interagem entre si e influenciam o desempenho do processo de solução. Tal discussão é feita na Seção 11.

2. Formulação do Problema de Programação Inteira Mista

Neste trabalho, considera-se o problema geral de Programação Inteira Mista, no qual se deseja otimizar uma função linear sujeita a restrições lineares e a condições de integralidade sobre parte (ou a totalidade) das variáveis. A formulação geral pode ser escrita como:

$$\min c^\top x$$

sujeito a

$$Ax \leq b,$$

$$\ell \leq x \leq u,$$

$$x_j \in \mathbb{Z}, \quad \forall j \in I,$$

$$x_j \in \mathbb{R}, \quad \forall j \notin I,$$

onde $x \in \mathbb{R}^n$ representa o vetor de variáveis de decisão, $c \in \mathbb{R}^n$ é o vetor de custos da função objetivo, $A \in \mathbb{R}^{m \times n}$ é a matriz de coeficientes das restrições, $b \in \mathbb{R}^m$ é o vetor de lados direitos (*rhs*), e $\ell, u \in (\mathbb{R} \cup \{-\infty, +\infty\})^n$ representam, respectivamente, os limites inferiores (*lower bound*) e superiores

(*upper bound*) das variáveis. O conjunto $I \subseteq \{1, \dots, n\}$ indica os índices das variáveis que devem assumir valores inteiros, enquanto as demais podem assumir valores contínuos.

Essa formulação engloba casos particulares importantes, como problemas puramente inteiros, problemas binários e problemas mistos com variáveis contínuas e inteiras. A principal dificuldade computacional decorre da presença das restrições de integralidade, que tornam o problema substancialmente mais difícil do que sua contraparte contínua.

2.1. Relaxação linear

Um passo fundamental na resolução de problemas de Programação Inteira Mista é a construção da relaxação linear. Nessa relaxação, as restrições de integralidade são removidas, permitindo que todas as variáveis assumam valores reais dentro dos limites impostos. A relaxação linear associada ao problema é dada por:

$$\begin{aligned} & \min \quad c^\top x \\ & \text{sujeito a} \\ & \quad Ax \leq b, \\ & \quad \ell \leq x \leq u, \\ & \quad x \in \mathbb{R}^n. \end{aligned}$$

Se a solução obtida for inteira nas componentes que exigem integralidade, então ela também é solução viável do problema original e, consequentemente, ótima. No entanto, em muitos casos, a solução ótima da relaxação linear apresenta valores fracionários em variáveis que deveriam ser inteiras, tornando necessário o uso de mecanismos adicionais para restaurar a integralidade.

2.2. Branch-and-Bound e Branch-and-Cut

O método de *Branch-and-Cut* combina duas ideias centrais para resolver o problema inteiro a partir de sua relaxação linear. A primeira é o *branching*, que particiona a região viável em subproblemas menores, normalmente por meio da imposição de restrições adicionais sobre variáveis fracionárias. Cada

subproblema corresponde a um nó da árvore de busca, e ramos que não podem produzir soluções melhores que a incumbente são podados.

A segunda é a geração de cortes. Um corte é uma desigualdade válida para o conjunto de soluções inteiras, mas violada por alguma solução fracionária da relaxação linear. Ao ser adicionado ao modelo, ele fortalece a relaxação, aproximando-a da região inteira viável e reduzindo o esforço de enumeração.

De forma resumida, o solver resolve inicialmente a relaxação linear do problema. Se a solução for inviável, o nó é podado; se for inteira, pode atualizar a incumbente; se for fracionária, podem ser aplicados cortes e/ou heurísticas. Caso o nó permaneça ativo e a solução continue fracionária, realiza-se o *branching*, gerando novos subproblemas. Esse processo se repete até que todos os nós relevantes tenham sido explorados ou podados.

3. Arquitetura do solver

A classe `BranchAndBound` orquestra todos os componentes do solver. Ela recebe, via construtor, os objetos plugáveis que definem o comportamento da busca:

- `strategy`: política de seleção de nós (`DepthFirst`, `BestFirst` ou `BreadthFirst`);
- `branch_rule`: regra de escolha da variável de branching (padrão: `FirstFractionalRule`);
- `direction`: ordem de inserção dos filhos (`CEIL_FIRST` ou `FLOOR_FIRST`);
- `heuristics`: lista de `PrimalHeuristic`;
- `cutting_planes`: lista de geradores de cortes;
- `presolver`: pré-processamento opcional.

Durante a inicialização, o construtor lê a instância com `gp.read()`, coleta metadados da instância (número de variáveis, restrições, densidade, tipos), executa o presolver se presente, e então **relaxa a integridade** de todas as variáveis binárias e inteiras, convertendo-as para `GRB.CONTINUOUS`. O resultado é armazenado em `_base_model`, que serve como modelo LP base copiado a cada nó. Os tipos originais, bounds e nomes são preservados em `_orig_vtype`, `_orig_lb`, `_orig_ub` e `_names`.

3.1. Métodos auxiliares

O método `_solve_lp(node)` copia `_base_model`, reaplica os bounds locais do nó (`lower_bounds` e `upper_bounds`), resolve a relaxação LP e preenche `node.lp_bound` e `node.lp_solution`. Retorna uma string entre "optimal", "infeasible" e "unbounded".

O método `_branch(node, var_idx)` cria dois nós filhos a partir do nó atual: `child_floor`, com `UB[var_idx] =  $\lfloor x_j^* \rfloor$` , e `child_ceil`, com `LB[var_idx] =  $\lceil x_j^* \rceil$` . Ambos herdam os bounds do pai por cópia de dicionário.

O método `_try_update_incumbent(solution, source)` recalcula o valor objetivo da solução candidata, compara com `best_obj` via `_is_better()` e, se houver melhora, atualiza `best_sol` e registra o evento em `stats["incumbents"]`.

Os métodos `_is_better()` e `_can_prune()` encapsulam a comparação de bounds respeitando o sentido do problema (minimização ou maximização), com tolerância `1e-6`.

Os métodos `_run_heuristics(node)` e `_run_cutting_planes(node)` iteram sobre as listas `self.heuristics` e `self.cutting_planes`, chamando `try_run()` em cada elemento e acumulando estatísticas de tempo, chamadas e acertos nos dicionários `heuristic_stats` e `cut_stats`.

3.2. Loop principal: `solve()`

O método `solve()` implementa o laço do *Branch-and-Bound*. A cada iteração, um nó é selecionado pela estratégia, sua relaxação LP é resolvida por `_solve_lp()`, e o fluxo segue a sequência abaixo:

1. **Poda por inviabilidade:** se o LP for inviável, o nó é descartado;
2. **Geração de cortes:** se `cutting_planes` não estiver vazia, chama `_run_cutting_planes()`; se cortes forem adicionados ao `_base_model`, o LP do nó é resolvido novamente;
3. **Heurísticas primais:** chama `_run_heuristics()` para tentar encontrar um incumbente;
4. **Poda por bound:** se `_can_prune(node.lp_bound)`, o nó é descartado;
5. **Integralidade:** se todas as variáveis inteiras estiverem integrais, atualiza o incumbente e continua;
6. **Branching:** escolhe a variável fracionária via `branch_rule.choose_variable()`, cria os dois filhos com `_branch()` e os insere na estrutura de nós abertos via `strategy.add_children()`.

Ao final do laço, `solve()` consolida todas as estatísticas em `self.stats` e retorna um dicionário com status, valor objetivo, solução e contadores. O método `export_stats()` serializa `self.stats` em JSON para análise posterior.

4. Presolve

Antes de iniciar o *Branch-and-Bound*, o solver pode executar uma etapa de *presolve* diretamente sobre o modelo original. O objetivo dessa etapa é simples: modificar o modelo antes da busca para reduzir o número de bounds frouxos, simplificar restrições e, quando possível, detectar inviabilidade sem precisar abrir a árvore. Nesta implementação, o *presolve* foi construído como um módulo independente e extensível.

4.1. Arquitetura do módulo

O núcleo do módulo é formado por três classes: `PresolveResult`, `PresolveRule` e `Presolver`.

A classe `PresolveResult` armazena, de forma padronizada, o efeito produzido por uma regra de *presolve*, incluindo indicadores de inviabilidade, número de bounds apertados, restrições removidas, variáveis fixas identificadas, alterações em coeficientes e tempo de execução.

A classe abstrata `PresolveRule` define a interface comum de todas as regras. Cada subclasse implementa o método

$$\text{apply}(\text{model}) \rightarrow \text{PresolveResult},$$

recebendo o modelo e podendo modificar diretamente bounds, lados direitos e, em algumas regras, remover ou recriar restrições.

A coordenação fica a cargo da classe `Presolver`. Seu método `run(model)` percorre a lista de regras em sequência, registra métricas antes e depois de cada aplicação, acumula o histórico de execução e interrompe imediatamente se alguma regra detectar inviabilidade. Assim, do ponto de vista do solver, o *presolve* é um pipeline linear de transformações aplicado ao modelo antes do início da árvore de busca.

4.2. Bound Tightening

A implementação do `BoundTightening` começa extraindo do modelo:

- a lista de variáveis;

- a lista de restrições;
- os bounds atuais em vetores mutáveis `lb` e `ub`;
- os tipos das variáveis em `vtype`;
- uma representação das linhas como listas de pares `(idx, coef)`.

Esse pré-processamento é importante porque evita chamadas repetidas à API do Gurobi dentro dos laços mais internos. Em vez de consultar o modelo a cada operação, a regra trabalha em estruturas Python já montadas e só escreve os bounds finais no modelo ao final da execução.

A propagação é feita por rodadas. Para cada restrição e para cada variável da linha, a implementação calcula a contribuição mínima e máxima das demais variáveis, armazenadas em `rest_min` e `rest_max`. A partir disso, deriva novos valores candidatos para `new_lb` e `new_ub`. Se a variável for inteira ou binária, esses valores são arredondados com `ceil` e `floor`. Se surgir uma inconsistência com

$$\text{new_lb} > \text{new_ub} + \text{eps},$$

a regra marca inviabilidade e retorna imediatamente. Caso contrário, os vetores locais de bounds são atualizados, e a regra continua até que nenhuma modificação ocorra em uma rodada ou até atingir `max_rounds`.

No final, os bounds acumulados são copiados de volta para os objetos `Var` do Gurobi e o modelo é atualizado com `model.update()`.

4.3. Row Feasibility and Redundancy

A regra `RowFeasibilityAndRedundancy` foi implementada como uma varredura simples sobre as restrições do modelo. Para cada restrição, a rotina lê a linha com `model.getRow(c)`, converte o sentido da desigualdade para a forma geral

$$\text{lhs} \leq \text{expr} \leq \text{rhs},$$

e calcula os valores `min_act` e `max_act` com base nos bounds atuais das variáveis.

A regra então faz duas verificações independentes. Primeiro, testa inviabilidade: se a atividade máxima ainda for insuficiente para satisfazer um lower bound da linha, ou se a atividade mínima já violar um upper bound, o

modelo é declarado inviável. Segundo, testa redundância: se todo o intervalo `[min_act, max_act]` já estiver contido no intervalo permitido pela restrição, a linha é marcada para remoção.

Na implementação, as restrições redundantes não são removidas imediatamente durante a iteração; elas são acumuladas em uma lista `redundant` e removidas em bloco ao final com `model.remove(redundant)`. Isso evita problemas de modificar a coleção de restrições enquanto ela está sendo percorrida.

4.4. *Substitute Fixed Variables*

A regra `SubstituteFixedVariables` começa identificando variáveis fixas por meio da condição

$$|UB - LB| \leq \text{eps}.$$

Essas variáveis são armazenadas em um dicionário, que associa o índice da variável ao seu valor fixado.

Em seguida, a implementação percorre todas as restrições e, para cada linha que contém ao menos uma variável fixa, calcula um *shift* correspondente à contribuição dessas variáveis na expressão linear. Esse valor é então usado para ajustar o lado direito da restrição por meio de

$$c.RHS = c.RHS - \text{shift}.$$

4.5. *Singleton Row Reduction*

A regra `SingletonRowReduction` atua sobre restrições em que `row_gurobi.size() == 1`. Na implementação, esse teste é usado diretamente para filtrar as chamadas.

Quando encontra uma linha singleton, a regra extrai a variável, seu coeficiente, o sentido da restrição e o lado direito. Em seguida, converte a linha para a forma com lower e upper bound implícitos sobre a variável. Dependendo do sinal do coeficiente, a direção do bound pode ser invertida. Para variáveis inteiras, os bounds derivados são arredondados.

Se o novo lower bound for maior do que o bound atual da variável, a implementação atualiza `v.LB`. Se o novo upper bound for menor, atualiza `v.UB`. Em seguida, testa inviabilidade local na própria variável. As restrições singleton processadas são armazenadas em uma lista e removidas ao final.

Essa implementação é curta e direta, mas é importante no módulo porque transforma linhas simples em bounds explícitos, reforçando o efeito de outras regras de presolve.

4.6. *Implicit Free Variable Detection*

A regra `ImplicitFreeVariableDetection` tem uma implementação mais analítica. Primeiro, ela pré-computa os dados de todas as linhas em `rows_data` e constrói um índice invertido `var_to_rows`, que associa cada variável às restrições em que ela aparece. Isso evita percorrer todas as restrições para cada variável.

Depois, para cada variável não fixa e com bounds explícitos finitos, a regra percorre apenas as linhas relevantes e tenta derivar um intervalo implicado pelas restrições, armazenado em `row_lb` e `row_ub`. Quando esse intervalo já é suficiente para implicar os bounds explícitos da variável, tais bounds são tratados como redundantes do ponto de vista estrutural. Opcionalmente, se `relax=True`, a regra relaxa esses bounds redundantes para variáveis contínuas, substituindo-os por $\pm\infty$. Para variáveis inteiras, o código apenas detecta essa situação, sem remover os bounds explicitamente.

4.7. *Euclidean Reduction*

A regra `EuclideanReduction` percorre as restrições verificando se todas as variáveis da linha são inteiras e se todos os coeficientes são inteiros dentro de uma tolerância. Quando isso ocorre, o código calcula o máximo divisor comum dos coeficientes por meio do método auxiliar `_gcd_list()`.

Para igualdades, a regra testa se o lado direito é divisível pelo MDC; se não for, detecta inviabilidade. Caso contrário, a restrição pode ser recriada com coeficientes e lado direito reduzidos pelo divisor comum. Para inequações, a implementação calcula um lado direito ajustado com `floor` ou `ceil`, conforme o sentido da restrição, e em alguns casos recria a linha reduzida, enquanto em outros apenas atualiza o RHS da restrição original. Portanto, trata-se de uma regra que pode tanto simplificar numericamente a linha quanto reforçar sua forma aritmética, embora a transformação aplicada dependa do caso tratado no código.

4.8. *Coefficient Strengthening*

A regra `CoefficientStrengthening` foi implementada apenas para restrições do tipo $\leq$. Para cada restrição elegível, o código extrai seus coeficientes e monta uma cópia mutável em `new_coeffs`, além de uma cópia do lado direito em `new_rhs`.

Em seguida, a rotina percorre as variáveis inteiras da linha. Para cada uma, calcula `rest_max`, isto é, a atividade máxima das demais variáveis, e aplica um critério algébrico para verificar se o coeficiente da variável pode ser

reduzido, com a devida compensação no lado direito, de forma a fortalecer a restrição segundo a regra implementada. Se pelo menos uma modificação ocorrer, a restrição antiga é removida e uma nova restrição linear é adicionada ao modelo com o mesmo nome.

Assim, a implementação adota a estratégia de recriação explícita da linha, em vez de alterar coeficientes diretamente sobre a restrição existente.

4.9. Chvátal-Gomory Strengthening

A regra `ChvatalGomoryStrengthening` foi implementada como um procedimento simplificado de fortalecimento inspirado em ideias de Chvátal-Gomory para restrições do tipo $\geq$ com variáveis inteiras não-negativas. O código extrai os coeficientes da linha, verifica elegibilidade e monta um conjunto finito de escalares candidatos com base em `amax`, `amin` e no parâmetro `max_scales`.

Para cada escalar candidato, a implementação calcula coeficientes e lado direito arredondados com `ceil`. Em seguida, avalia se a nova linha é estritamente mais forte do que a original segundo um critério simples e atribui um *score* baseado na razão entre a soma dos coeficientes e o lado direito. A melhor alternativa encontrada é então usada para substituir a restrição original.

Em termos de implementação, trata-se de uma busca discreta sobre um conjunto pequeno de transformações possíveis da linha original, e não de uma rotina geral de geração de cortes de Chvátal-Gomory.

4.10. Parallel Rows

A regra `ParallelRows` trabalha em duas etapas. Primeiro, extrai de cada restrição um vetor ordenado de coeficientes e o intervalo correspondente `[lhs, rhs]`. Depois, compara pares de restrições por meio do método auxiliar `_proportionality_factor()`, que testa se os vetores de coeficientes são proporcionais e, em caso positivo, retorna o fator λ .

Se duas linhas forem paralelas, a implementação normaliza os bounds de uma delas pela razão λ , calcula a interseção dos intervalos e verifica se ela é vazia. Se for, a regra detecta inviabilidade. Caso contrário, uma das restrições é marcada para remoção e a outra tem seus limites lógicos atualizados internamente.

Na etapa final, entretanto, a atualização da restrição mantida é feita de forma simplificada, pois o tratamento completo de restrições rangeadas exigiria uma representação adicional. Assim, quando o intervalo combinado não

pode ser representado exatamente pela forma original da restrição, o código preserva apenas o lado mais restritivo compatível com o sentido disponível no modelo.

4.11. Orquestração pelo *Presolver*

A classe `Presolver` é pequena, mas importante para a integração do módulo. Seu construtor recebe uma lista de regras, com `BoundTightening()` como padrão. O método `run(model)` executa cada regra em sequência, mede seu tempo de execução, registra métricas estruturadas em um histórico e acumula estatísticas agregadas.

Se alguma regra retorna `infeasible=True`, o método encerra imediatamente e devolve a tupla `(False, history)`. Caso contrário, ao final devolve `(True, history)`, indicando que o modelo pode seguir para a fase de resolução. Na prática, isso encapsula todo o *presolve* em um único ponto de entrada, facilitando sua ativação no solver principal.

4.12. Síntese

O aperto de bounds aparece principalmente em `BoundTightening` e `SingletonRowReduction`; a remoção ou simplificação de restrições triviais aparece em `RowFeasibilityAndRedundancy`, `SubstituteFixedVariables`, `SingletonRowReduction` e `ParallelRows`; a detecção simples de inviabilidade está distribuída em várias regras; e a atualização dos limites das variáveis ocorre diretamente sobre o modelo original antes do início da árvore.

Além disso, a implementação foi além do núcleo mínimo, incorporando regras adicionais como `ImplicitFreeVariableDetection`, `EuclideanReduction`, `CoefficientStrengthening` e `ChvatalGomoryStrengthening`. O resultado é um módulo consistente com o restante da arquitetura do solver: cada regra é independente, opera in-place sobre o modelo e pode ser combinada de forma flexível dentro do pipeline de pré-processamento.

5. Busca em árvore

A etapa de busca em árvore é a base do *Branch-and-Bound*. Quando a solução da relaxação linear de um nó é fracionária, o problema é particionado em subproblemas menores por meio de *branching*, originando uma árvore de busca. A forma como os nós abertos são armazenados e selecionados afeta diretamente o número de nós explorados, o tempo de execução e a velocidade para encontrar incumbentes.

Nesta implementação, a busca em árvore é estruturada por meio de duas abstrações: a classe `Node`, que representa um nó da árvore, e a interface abstrata `NodeSelection`, que encapsula a política de exploração. Essa separação permite trocar a estratégia de busca sem alterar a lógica principal do solver.

5.1. Estrutura de um nó da árvore

Cada nó da árvore é representado pela classe `Node` e corresponde a um subproblema obtido a partir do problema original pela adição de limites locais sobre variáveis inteiras. Na implementação, cada nó armazena:

- `lower_bounds` e `upper_bounds`: dicionários com os limites locais impostos pelo *branching* ao longo do caminho da raiz até o nó;
- `depth`: profundidade do nó na árvore;
- `lp_bound` e `lp_solution`: valor ótimo e solução da relaxação linear do nó;
- metadados sobre o *branching* do pai, usados para atualização de pseudocustos.

Se em algum nó a solução da relaxação linear produz $x_j^* \notin \mathbb{Z}$ para uma variável inteira x_j , o *branching* gera dois filhos com os limites

$$x_j \leq \lfloor x_j^* \rfloor \quad \text{e} \quad x_j \geq \lceil x_j^* \rceil,$$

armazenados em `upper_bounds` e `lower_bounds` dos respectivos nós filhos. A árvore é construída implicitamente: o solver copia o modelo relaxado e aplica os *bounds* locais antes de resolver cada LP, sem duplicar a formulação completa.

A classe `Node` implementa ainda dois métodos auxiliares: `is_integer_feasible`, que verifica se todos os valores das variáveis inteiras na solução LP são inteiros dentro de uma tolerância, e `first_fractional`, que retorna a primeira variável inteira fracionária da solução.

5.2. Abstração *NodeSelection*

A política de exploração da árvore foi encapsulada na classe abstrata `NodeSelection`, que define quatro operações: `initialize()`, `add_node()`, `add_children()` e `select_node()`. Essa abstração separa duas decisões distintas do solver: qual nó explorar em seguida, responsabilidade de `NodeSelection`, e qual variável usar para *branching*, responsabilidade de `BranchVariableRule`.

Após selecionar um nó e resolver sua relaxação linear, o solver pode encerrá-lo por três motivos: inviabilidade da LP, integralidade da solução, ou poda por bound. No caso de minimização, a poda ocorre quando

$$z_{LP}(N) \geq z^{\text{inc}} - \varepsilon,$$

onde $z_{LP}(N)$ é o valor da relaxação linear do nó N e z^{inc} é o valor do incumbente atual. Em maximização, a desigualdade é invertida.

5.3. Estratégia *Depth-First Search*

A estratégia `DepthFirst` implementa a busca em profundidade. Os nós abertos são armazenados em uma lista Python usada como pilha LIFO: `add_node()` e `add_children()` inserem com `append()`, e `select_node()` remove com `pop()`.

A DFS prioriza o aprofundamento em um ramo antes de explorar alternativas. Em geral, consome pouca memória e tende a encontrar incumbentes rapidamente, mas pode explorar ramos ruins por muito tempo antes de visitar regiões mais promissoras. A ordem de exploração dos filhos respeita o parâmetro `BranchDirection`: se `CEIL_FIRST`, o filho $[x_j^*]$ é inserido por último na pilha e, portanto, explorado primeiro.

5.4. Estratégia *Breadth-First Search*

A estratégia `BreadthFirst` implementa a busca em largura. Os nós abertos são armazenados em uma `collections.deque` usada como fila FIFO: `add_node()` e `add_children()` inserem com `append()`, e `select_node()` remove com `popleft()`.

A BFS percorre a árvore por níveis, processando todos os nós de uma profundidade antes de avançar para a seguinte. Tende a consumir mais memória do que a DFS e costuma ser menos eficiente na prática, mas é conceitualmente simples e útil para análises estruturais da árvore.

5.5. Estratégia Best-First Search

A estratégia **BestFirst** seleciona, entre todos os nós abertos, aquele cujo *bound* LP é mais promissor. Em minimização, escolhe o nó com menor *bound*; em maximização, o de maior *bound*:

$$N^* \in \arg \min_{N \in \mathcal{O}} z_{LP}(N) \quad (\text{minimização}), \quad N^* \in \arg \max_{N \in \mathcal{O}} z_{LP}(N) \quad (\text{maximização}).$$

Na implementação, os nós abertos são armazenados em uma lista. O método `select_node()` percorre todos os nós, identifica o índice do melhor *bound* com `min()` ou `max()`, troca esse nó com o último elemento da lista e o remove com `pop()`.

A Best-First concentra o esforço nas regiões mais promissoras da árvore e tende a reduzir o número total de nós explorados. Em contrapartida, pode demorar mais para encontrar um incumbente inicial e exige comparar continuamente todos os nós abertos a cada seleção.

6. Seleção de Variável para Branching

Após resolver a relaxação linear de um nó da árvore, caso a solução obtida seja fracionária, o solver precisa escolher em qual variável inteira realizar o próximo *branching*. Essa decisão é um dos elementos centrais do desempenho de algoritmos de *Branch-and-Bound* e *Branch-and-Cut*, pois influencia diretamente o formato da árvore, a qualidade dos *bounds* obtidos nos filhos e o número total de nós explorados.

De forma geral, seja x^* a solução ótima da relaxação linear de um nó. Se existe uma variável inteira x_j tal que

$$x_j^* \notin \mathbb{Z},$$

então o solver precisa escolher um índice j e criar dois subproblemas:

$$x_j \leq \lfloor x_j^* \rfloor \quad \text{e} \quad x_j \geq \lceil x_j^* \rceil.$$

A qualidade dessa escolha depende de quão bem ela reduz o espaço viável fracionário e de quão rapidamente os subproblemas resultantes podem ser podados. Na implementação desenvolvida, essa decisão foi encapsulada em uma hierarquia de classes específica, separada da estratégia de seleção de nós da árvore.

6.1. Abstração *BranchVariableRule*

A seleção da variável de branching foi modelada por meio da classe abstrata **BranchVariableRule**. Essa classe define a interface comum para qualquer regra de escolha de variável, de forma análoga ao que foi feito para as estratégias de busca em árvore.

A interface possui como método principal:

- **choose_variable(node, bb)**: recebe o nó atual e o contexto do solver, retornando o índice de uma variável inteira fracionária a ser ramificada.

Além disso, a classe oferece dois *hooks* opcionais:

- **on_node_solved(node, bb)**: pode ser usado para atualizar informações após a solução LP de um nó;
- **on_branch_result(...)**: permite atualizar estatísticas com base no resultado dos filhos de um branching real, sendo especialmente útil para pseudocustos.

Essa abstração torna a arquitetura modular, isto é, a lógica do solver apenas pergunta qual variável deve ser escolhida, sem depender do critério específico adotado. Assim, regras simples, como selecionar a primeira variável fracionária, e regras mais sofisticadas, como *Strong Branching* ou *Pseudocost Branching*, podem ser usadas de forma intercambiável.

6.2. Regra *FirstFractionalRule*

A regra **FirstFractionalRule** é a mais simples entre as implementadas. Ela percorre o conjunto de variáveis inteiras do modelo e retorna a primeira variável cuja solução no nó atual seja fracionária dentro de uma tolerância numérica.

Formalmente, dado o conjunto I de variáveis inteiras, a regra escolhe o primeiro índice $j \in I$ tal que

$$|x_j^* - \lfloor x_j^* \rfloor| > \varepsilon,$$

onde ε é uma tolerância pequena.

Essa estratégia não utiliza nenhuma informação adicional sobre qualidade do bound, impacto do branching ou histórico da busca. Seu principal papel

é servir como *baseline* simples e de baixo custo computacional. Como vantagem, tem-se que o custo é praticamente nulo para escolher a variável, mas a escolha depende apenas da ordem das variáveis no modelo, podendo levar a árvores ruins.

Apesar de simples, essa regra é útil como referência experimental e também como mecanismo de fallback quando regras mais sofisticadas não conseguem produzir uma escolha.

6.3. Regra *MostFractionalRule*

A regra **MostFractionalRule** busca selecionar a variável mais fracionária da solução LP. A intuição é que valores próximos de meio inteiro, isto é, mais distantes dos inteiros vizinhos, tendem a representar casos em que a integralidade está mais fortemente violada, tornando o branching mais efetivo.

Se x_j^* é o valor fracionário de uma variável inteira, define-se sua parte fracionária por

$$\text{frac}(x_j^*) = x_j^* - \lfloor x_j^* \rfloor.$$

A medida de fracionaridade usada na implementação é

$$s_j = \min(\text{frac}(x_j^*), 1 - \text{frac}(x_j^*)).$$

Esse valor mede a distância até o inteiro mais próximo. Quanto maior s_j , mais “fracionária” é a variável, atingindo o máximo em valores do tipo $k + 0.5$. A regra então escolhe

$$j^* \in \arg \max_{j \in I_f} s_j,$$

onde I_f é o conjunto das variáveis inteiras fracionárias no nó atual.

Na implementação, a regra percorre todas as variáveis inteiras, ignora as que já são inteiras dentro da tolerância, calcula o *score* s_j e mantém a variável com maior valor. Caracteriza-se por combinar baixo custo computacional e uso explícito da informação da solução LP.

6.4. Regra *StrongBranchingRule*

A regra **StrongBranchingRule** é significativamente mais sofisticada. Em vez de decidir com base apenas no valor fracionário de cada variável, ela testa explicitamente o efeito de ramificar em algumas candidatas antes de realizar o branching real. A ideia é estimar qual variável produzirá a maior

degradação do bound nos filhos, o que tende a reduzir o tamanho total da árvore.

Considere um nó com bound LP z_P e uma variável inteira fracionária x_j^* . O *Strong Branching* avalia temporariamente os dois subproblemas:

$$x_j \leq \lfloor x_j^* \rfloor \quad \text{e} \quad x_j \geq \lceil x_j^* \rceil.$$

Se os valores das relaxações lineares desses filhos forem $z_j^\downarrow$ e $z_j^\uparrow$, respectivamente, então, em um problema de minimização, os ganhos de bound são dados por

$$g_j^\downarrow = z_j^\downarrow - z_P, \quad g_j^\uparrow = z_j^\uparrow - z_P.$$

Quanto maiores esses valores, melhor, pois isso significa que os filhos são menos promissores e, portanto, mais próximos de poda por bound. Se um dos filhos for inviável, a implementação atribui a ele um ganho muito grande, refletindo o fato de que esse ramo seria imediatamente podado.

A partir desses ganhos, a regra combina os dois lados por meio de um *score*. A implementação suporta três modos:

- **product:**

$$s_j = \max(0, g_j^\downarrow) \cdot \max(0, g_j^\uparrow),$$

favorecendo variáveis que tornam ambos os lados fortes;

- **min:**

$$s_j = \min(g_j^\downarrow, g_j^\uparrow),$$

maximizando o pior caso;

- **sum:**

$$s_j = g_j^\downarrow + g_j^\uparrow,$$

privilegiando o ganho total agregado.

A regra então escolhe

$$j^* \in \arg \max_j s_j.$$

Como resolver LPs temporários para todas as variáveis fracionárias seria caro, a implementação primeiro constrói uma lista de candidatas, ordenadas

por fracionaridade, e limita a avaliação a um número máximo `max_candidates`. Assim, o custo do *Strong Branching* permanece controlado.

Em termos práticos, essa regra tende a produzir árvores menores, pois escolhe variáveis com maior impacto nos bounds. Contudo, seu custo por nó é substancialmente maior, já que requer resolver relaxações LP adicionais antes do branching efetivo. Em consequência, o ganho em número de nós nem sempre se traduz em menor tempo total, especialmente em instâncias maiores.

6.5. Regra *PseudoCostRule*

A regra `PseudoCostRule` procura capturar os benefícios do *Strong Branching* com custo computacional muito menor. Em vez de resolver LPs adicionais em cada decisão, ela mantém estimativas históricas do ganho esperado ao ramificar em cada variável, chamadas de pseudocustos.

Suponha que, em algum nó, a variável x_j tenha valor fracionário $x_j^* = v$. Define-se a distância até o ramo inferior por

$$\Delta_j^\downarrow = v - \lfloor v \rfloor,$$

e a distância até o ramo superior por

$$\Delta_j^\uparrow = \lceil v \rceil - v.$$

Sempre que um branching real é executado e os filhos são resolvidos, a implementação observa os ganhos efetivos de bound e calcula pseudocustos por unidade de distância:

$$pc_j^\downarrow = \frac{g_j^\downarrow}{\Delta_j^\downarrow}, \quad pc_j^\uparrow = \frac{g_j^\uparrow}{\Delta_j^\uparrow}.$$

Ao longo da busca, esses valores são acumulados e promediados. Sejam $\bar{p}_j^\downarrow$ e $\bar{p}_j^\uparrow$ os pseudocustos médios armazenados para a variável j . Então, em um nó futuro com valor fracionário v , os ganhos estimados são

$$\hat{g}_j^\downarrow = \bar{p}_j^\downarrow \cdot \Delta_j^\downarrow, \quad \hat{g}_j^\uparrow = \bar{p}_j^\uparrow \cdot \Delta_j^\uparrow.$$

Esses ganhos estimados são combinados por uma função de score análoga à do *Strong Branching*:

- **product:**

$$s_j = \max(0, \hat{g}_j^\downarrow) \cdot \max(0, \hat{g}_j^\uparrow);$$

- **min:**

$$s_j = \min(\hat{g}_j^\downarrow, \hat{g}_j^\uparrow);$$

- **sum:**

$$s_j = \hat{g}_j^\downarrow + \hat{g}_j^\uparrow.$$

A variável escolhida é aquela com maior score previsto.

Um aspecto importante da implementação é a noção de *reliability*. Antes que uma variável seja considerada confiável para pseudocustos, é necessário que ela tenha sido observada um número mínimo de vezes em cada direção de branching. Enquanto esse histórico ainda não existe, a regra recorre a uma forma limitada de *Strong Branching* em um pequeno subconjunto de candidatas para inicializar boas estimativas.

Essa estratégia combina duas fases. Na fase inicial, coleta informação por meio de avaliações fortes em poucas variáveis. Já na fase madura, passa a usar apenas pseudocustos médios para decidir rapidamente.

Com isso, a **PseudoCostRule** busca o melhor dos dois mundos, decisões informadas por impacto esperado, mas sem o custo de resolver LPs temporários a cada nó.

6.6. Atualização dos pseudocustos

Na implementação, a atualização dos pseudocustos ocorre após a resolução de nós filhos reais da árvore. Para cada filho resolvido, o solver conhece:

- o bound do nó pai;
- a variável usada no branching;
- o lado do branching (**floor** ou **ceil**);
- o valor fracionário da variável no nó pai;
- o bound LP do nó filho.

A partir desses dados, o ganho observado é calculado e normalizado pela distância correspondente. Os acumuladores **down_sum**, **down_count**, **up_sum** e **up_count** armazenam os totais históricos por variável. Assim, os pseudocustos médios são recalculados implicitamente por

$$\bar{p}_j^\downarrow = \frac{\text{down_sum}[j]}{\text{down_count}[j]}, \quad \bar{p}_j^\uparrow = \frac{\text{up_sum}[j]}{\text{up_count}[j]}.$$

Esse mecanismo torna a regra adaptativa: à medida que a árvore é explorada, o solver aprende quais variáveis tendem a produzir branchings mais fortes.

6.7. Integração com o solver

No motor principal do solver, a escolha da variável de branching ocorre somente após:

1. resolver a relaxação linear do nó;
2. eventualmente aplicar cortes;
3. executar heurísticas primais;
4. verificar poda por inviabilidade, integralidade ou bound.

Se o nó permanece fracionário, o método

`branch_rule.choose_variable(node, self)`

é chamado. Caso a regra não retorne nenhuma variável, o solver utiliza `first_fractional` como fallback. Em seguida, o método de branching cria os dois filhos

$$x_j \leq \lfloor x_j^* \rfloor \quad \text{e} \quad x_j \geq \lceil x_j^* \rceil,$$

que são inseridos na estrutura de nós abertos segundo a estratégia de busca escolhida.

7. Geração de cortes

Cortes são desigualdades adicionadas ao modelo que eliminam a solução fracionária atual sem remover nenhuma solução inteira viável. Diferentemente do presolve, que atua antes do *Branch-and-Bound*, os cortes operam após a resolução da relaxação LP de um nó, usando informação da base ótima e das restrições do modelo.

7.1. Arquitetura geral

Todas as famílias de cortes herdam da classe abstrata `CuttingPlane`, que centraliza o controle de execução e a infraestrutura compartilhada de extração e inserção. Os principais parâmetros são `only_root`, `max_cuts`, `tol` e `probability`.

O método `try_run(node, bb)` verifica elegibilidade via `should_run()` e aplica o sorteio probabilístico antes de chamar `run()`. Para as classes baseadas em tableau, `run()` segue o fluxo: extrair linhas fracionárias com `_extract_fractional_rows()`, chamar `generate_cut()` em cada linha, converter o corte para o espaço das variáveis estruturais e adicioná-lo ao modelo com `_add_cut_to_model()`. Essa separação permite reutilizar toda a infraestrutura de extração do tableau entre diferentes famílias, mudando apenas a fórmula de geração.

Por padrão, os cortes usam `only_root=True`. Um corte derivado do tableau de um nó interno depende dos *bounds* locais impostos pelo *branching* e, em geral, é válido apenas naquela subárvore. Na raiz, ainda não existem *bounds* locais, de modo que o corte é válido para o poliedro inteiro original e pode ser adicionado permanentemente a `bb._base_model`, sendo reaproveitado por todos os nós futuros.

7.2. Estrutura auxiliar `_CutRow` e extração do tableau

Para os cortes baseados em tableau, a informação relevante é encapsulada na estrutura `_CutRow`, que armazena a coluna básica fracionária, o valor b_i e sua parte fracionária f_0 , as entradas não-básicas da linha, informações de largura de *bounds* e a matriz estrutural A e vetor b usados na eliminação de folgas.

Como o Gurobi não expõe diretamente o tableau simplex, o método `_extract_fractional_rows()` o reconstrói a partir da base ótima: copia o modelo, aplica os *bounds* locais do nó, resolve a LP, identifica colunas básicas e não-básicas via `VBasis` e `CBasis`, monta a matriz de base B , calcula B^{-1} via `numpy` e reconstrói os coeficientes das colunas não-básicas em cada linha. Apenas linhas cuja variável básica seja inteira e fracionária são retornadas.

7.3. Reconstrução do corte no espaço original

Os cortes de MIG e *Lifting & Extending* são obtidos em termos de variáveis deslocadas w_j , medidas a partir do *bound* ativo. O método `_finalize_cut()`

converte esses coeficientes para o espaço das variáveis estruturais x_j , tratando os dois casos:

$$w_j = x_j - \ell_j \quad \text{ou} \quad w_j = u_j - x_j.$$

Variáveis de folga são eliminadas substituindo $s_i = b_i - \sum_k A_{ik} x_k$. O método retorna os coeficientes em x_j , o lado direito e o sentido da desigualdade.

7.4. *Mixed-Integer Gomory Cut*

A classe `MixedIntegerGomoryCut` implementa o corte de Gomory misto-inteiro clássico. A partir de uma linha do tableau

$$x_B + \sum_j a_j w_j = b_i,$$

com $f_0 = \text{frac}(b_i)$, o coeficiente α_j de cada variável não-básica é calculado pelo método estático `_gmi_alpha(a, f0, kind)`:

- variável inteira: $f_j = \text{frac}(a_j)$; $\alpha_j = f_j/f_0$ se $f_j \leq f_0$, caso contrário $(1 - f_j)/(1 - f_0)$;
- variável contínua: $\alpha_j = a_j/f_0$ se $a_j \geq 0$, caso contrário $-a_j/(1 - f_0)$.

A desigualdade resultante é $\sum_j \alpha_j w_j \geq 1$, que é então convertida para o espaço original por `_finalize_cut()`.

7.5. *MIR*

A classe `SimpleMIR` implementa o corte MIR (*Mixed-Integer Rounding*) trabalhando diretamente sobre as restrições originais do modelo, sem usar o tableau. O método `run()` resolve a LP do nó e percorre as restrições, aplicando filtros conservadores: apenas restrições $\leq$, lado direito fracionário, variáveis com $LB \geq 0$. Para cada linha elegível, os coeficientes são transformados pela fórmula MIR, distinguindo variáveis inteiras e contínuas. Apenas cortes efetivamente violados pela solução LP atual são adicionados ao modelo base.

7.6. *Lifting & Extending*

A classe `LiftingExtending` fortalece o corte MIG para variáveis com *bounds* finitos dos dois lados. O método `generate_cut()` reutiliza `_gmi_alpha()` do MIG, mas para cada variável não-básica com largura de *bound* finita calcula também o coeficiente complementar, obtido invertendo o lado do *bound* ativo ($a_j \rightarrow -a_j$). Entre os dois coeficientes, mantém o maior: na forma normalizada $\sum_j \alpha_j w_j \geq 1$, um coeficiente maior produz uma desigualdade mais forte. Quando a largura do *bound* é infinita, o resultado colapsa no MIG padrão.

7.7. *Cover Cut*

A classe `CoverCut` implementa separação de cortes de cobertura para restrições do tipo mochila binária,

$$\sum_j a_j x_j \leq b, \quad x_j \in \{0, 1\}.$$

A classe busca subconjuntos C tais que $\sum_{j \in C} a_j > b$, o que implica a desigualdade válida $\sum_{j \in C} x_j \leq |C| - 1$.

O método `run()` filtra restrições com coeficientes positivos e variáveis binárias. A separação é feita por `_find_minimal_cover()`, que resolve uma relaxação linear do problema de separação via `scipy.optimize.linprog`, aplica um arredondamento guloso e poda o resultado para obter uma cobertura minimal. O corte só é aceito se violado pela solução LP atual.

7.8. *Integração com o solver*

Os geradores de cortes são armazenados em `cutting_planes` no solver principal. Após resolver a LP de um nó, cada gerador é chamado via `try_run(node, bb)`, que retorna o número de cortes adicionados. Como os cortes são incorporados diretamente a `bb._base_model`, todas as resoluções LP subsequentes já consideram a relaxação fortalecida, o que é especialmente relevante na raiz, onde cortes de qualidade podem reduzir significativamente a necessidade de *branching*.

8. Heurísticas primais

Uma solução inteira viável fornece um incumbente e, com isso, um limitante primal que permite podar nós por *bound*. Por esse motivo, o solver

implementa um conjunto amplo de heurísticas primais, todas integradas ao fluxo principal do *Branch-and-Bound* por meio de uma hierarquia modular e plugável.

Todas as heurísticas herdam da classe abstrata `PrimalHeuristic`, que centraliza três responsabilidades: verificar elegibilidade via `should_run()`, controlar ativação probabilística via `try_run()`, e delegar a execução ao método abstrato `run()`. O método `try_run()` evita duplicação de lógica nas subclasses: consulta `should_run()`, aplica o sorteio com `probability` e só então chama `run()`. No solver, as heurísticas são executadas em `_run_heuristics(node)`, que percorre `self.heuristics`, contabiliza chamadas e acertos, e tenta atualizar o incumbente com qualquer solução viável retornada.

O conjunto implementado inclui `RoundingHeuristic`, a família `DivingHeuristic` com subclasses `FractionalDiving`, `GuidedDiving` e `LockDiving`, além de `FeasibilityPump`, `FeasibilityJump`, `FixAndPropagate`, `RENS`, `RINS`, `LocalSearch`, `LocalBranching`, `LNS` e `ALNS`.

8.1. *Rounding Heuristic*

A `RoundingHeuristic` cria uma cópia de `bb._base_model`, aplica os *bounds* locais do nó e fixa cada variável inteira ao inteiro mais próximo, definindo simultaneamente $LB = UB = \text{round}(x_j^*)$, truncado aos *bounds* vigentes. O modelo resultante é então resolvido como LP; se o status for `GRB.OPTIMAL`, a solução é retornada.

8.2. *Família DivingHeuristic*

O *diving* parte da solução LP fracionária e vai fixando variáveis inteiras uma a uma, reotimizando a relaxação após cada fixação, até atingir integralidade ou inviabilidade. A classe abstrata `DivingHeuristic` implementa todo esse esqueleto em `run()`: copia os *bounds* do nó, itera até `max_depth`, chama a subclasse para escolher variável e valor, fixa com `lb = ub = value`, cria uma nova cópia do modelo, reaplica os *bounds* acumulados e resolve a LP. Se a LP ficar inviável, retorna `None`; se não houver mais variáveis fracionárias, retorna a solução. Cada subclasse implementa apenas `select_variable()` e `choose_value()`.

- **FractionalDiving:** escolhe a variável menos fracionária, $j^* \in \arg \min_{j \in I_f} |x_j^* - \text{round}(x_j^*)|$, e fixa no inteiro mais próximo.

- **GuidedDiving**: quando há incumbente, escolhe a variável com maior desacordo entre LP e incumbente, $j^* \in \arg \max_{j \in I_f} |x_j^* - \bar{x}_j|$, e fixa no valor do incumbente. Sem incumbente, recai automaticamente no comportamento da **FractionalDiving**.
- **LockDiving**: pré-computa, via `_compute_locks()`, quantas restrições são apertadas ao aumentar ou diminuir cada variável inteira. O resultado é armazenado em cache em `_lock_cache`. A variável escolhida é aquela com menor $\min(L_j^\uparrow, L_j^\downarrow)$, e o valor é fixado na direção de menor lock.

8.3. Feasibility Pump

A **FeasibilityPump** alterna entre arredondar a solução LP e resolver um LP auxiliar que minimiza a distância L_1 até essa projeção. A cada iteração, constrói $\bar{x}$ com o arredondamento das variáveis inteiras, cria uma cópia do modelo com variáveis auxiliares $t_j \geq 0$ satisfazendo

$$t_j \geq x_j - \bar{x}_j, \quad t_j \geq \bar{x}_j - x_j,$$

e substitui o objetivo por $\min \sum_{j \in I} t_j$. A heurística só é elegível quando não há incumbente.

8.4. Feasibility Jump

A **FeasibilityJump** estende a **FeasibilityPump** com detecção de ciclagem. Um conjunto `seen_projections` registra as projeções inteiras já visitadas. Quando uma projeção se repete, aplica uma perturbação aleatória em $\lfloor n/5 \rfloor$ variáveis inteiras via `random.sample()`, invertendo o arredondamento de cada variável perturbada. Após a perturbação, o mesmo LP auxiliar de distância L_1 é resolvido novamente.

8.5. Fix and Propagate

A **FixAndPropagate** mantém dicionários locais de *bounds*, coleta as variáveis inteiras fracionárias, ordena por fracionaridade crescente e fixa a mais confiável ao inteiro mais próximo, truncado aos *bounds* vigentes. Depois resolve novamente a LP com todos os *bounds* acumulados e propaga o efeito sobre as demais variáveis. O processo se repete até integralidade ou inviabilidade.

8.6. RENS

A **RENS** define uma vizinhança a partir da solução LP: variáveis já inteiras são fixadas exatamente, e variáveis fracionárias têm seus *bounds* restritos a $[\lfloor x_j^* \rfloor, \lceil x_j^* \rceil]$. O método `_solve_submip()` cria uma cópia do modelo, restaura os tipos inteiros originais, aplica essas restrições e resolve o subproblema como MIP com limite de tempo. Se falhar, novas tentativas são feitas fixando frações crescentes das variáveis fracionárias mais próximas de inteiro, controladas por `adaptive_ratios`.

8.7. RINS

A **RINS** combina a solução LP do nó com o incumbente. Variáveis em que $|x_j^* - \bar{x}_j| \leq \tau$ são fixadas ao incumbente; as demais formam a vizinhança livre, ranqueadas por

$$\text{score}(j) = \frac{|x_j^* - \bar{x}_j|}{|rc_j| + \varepsilon},$$

onde rc_j é o custo reduzido obtido por `_get_reduced_costs()`. Sub-MIPs são resolvidos com fixing adaptativo análogo ao **RENS**. A heurística não roda sem incumbente.

8.8. Local Search

A **LocalSearch** ranqueia as variáveis inteiras por $\text{score}(j) = |x_j^* - \bar{x}_j| + \beta |rc_j|$ e percorre janelas deslizantes de tamanho `window_size`. Em cada iteração, apenas as variáveis da janela ficam livres; as demais são fixadas ao incumbente. Cada janela gera um sub-MIP independente via `_solve_window()`.

8.9. Local Branching

A **LocalBranching** define a vizinhança do incumbente por distância de Hamming sobre variáveis binárias. Variáveis inteiras gerais em que LP e incumbente concordam são pré-fixadas para reduzir o sub-MIP. A restrição de vizinhança é inserida explicitamente:

$$\sum_{j: \bar{x}_j=1} (1 - x_j) + \sum_{j: \bar{x}_j=0} x_j \leq k.$$

Se `add_obj_cut=True`, adiciona também uma restrição de melhoria estrita no objetivo. Requer incumbente com variáveis binárias.

8.10. LNS

A LNS segue o paradigma *destroy-and-repair*. A hierarquia `LNSDestroyRule` encapsula a escolha do conjunto livre F , com quatro implementações: `RandomDestroy` (amostragem aleatória), `FractionalGuidedDestroy` (maior desacordo LP/incumbente), `ConstraintBasedDestroy` (variáveis nas restrições mais apertadas) e `ObjectiveBasedDestroy` (maior custo reduzido). O método `_repair()` fixa as variáveis fora de F ao incumbente e resolve o sub-MIP. O loop principal adapta o `destroy_rate`: reduz após melhora (intensificação) e aumenta após `patience` iterações sem melhora (diversificação).

8.11. ALNS

A ALNS generaliza a LNS com seleção adaptativa dos operadores de destruição. A classe mantém pesos adaptativos por operador, atualizados via suavização exponencial por segmentos:

$$w_k^{t+1} = \max(w_{\min}, (1 - \eta) w_k^t + \eta \sigma_k^t),$$

onde $\sigma_k^t \in \{\sigma_1, \sigma_2, \sigma_3, 0\}$ conforme o resultado da iteração. O operador é selecionado por roleta proporcional aos pesos via `_roulette_select()`. O critério de aceitação é configurável pelo enum `ALNSAcceptance`: guloso (`GREEDY`), por limiar (`THRESHOLD`) ou por *simulated annealing* (`SIMULATED_ANNEAL`), com temperatura inicializada automaticamente e resfriada por `temp_decay` a cada iteração.

9. Linearização por partes via SOS2

Uma abordagem clássica para tratar funções não lineares em modelos de programação inteira mista consiste em aproximá-las por funções lineares por partes (*piecewise linear*, PWL) e codificar a seleção do segmento ativo por meio de restrições SOS2 (*Special Ordered Sets of type 2*). Essa estratégia permite que o problema resultante seja resolvido diretamente por um solver MILP, sem necessidade de extensões não lineares.

9.1. Estrutura da aproximação

Dados K breakpoints $\bar{x}_0 < \bar{x}_1 < \dots < \bar{x}_{K-1}$ e os valores correspondentes $\bar{y}_i = f(\bar{x}_i)$, qualquer ponto (x, y) sobre a função linear por partes pode ser escrito como combinação convexa dos breakpoints:

$$x = \sum_{i=0}^{K-1} \bar{x}_i \lambda_i, \quad y = \sum_{i=0}^{K-1} \bar{y}_i \lambda_i,$$

sujeito a

$$\sum_{i=0}^{K-1} \lambda_i = 1, \quad \lambda_i \geq 0 \quad \forall i.$$

A condição SOS2 exige que no máximo dois λ_i consecutivos sejam não nulos simultaneamente, garantindo que o ponto (x, y) pertença a exatamente um segmento $[\bar{x}_i, \bar{x}_{i+1}]$.

9.2. Implementação via variáveis binárias

A função `add_sos2_pwl_gurobi()` implementa essa codificação diretamente sobre um modelo Gurobi, introduzindo dois conjuntos de variáveis auxiliares:

- $\lambda_i \in [0, 1]$, contínuas, para $i = 0, \dots, K - 1$: pesos da combinação convexa;
- $s_j \in \{0, 1\}$, binárias, para $j = 0, \dots, K - 2$: indicadoras do segmento ativo.

As restrições adicionadas ao modelo são:

$$\sum_i \lambda_i = 1 \quad (\text{convexidade}) \quad (1)$$

$$x = \sum_i \bar{x}_i \lambda_i \quad (\text{ligação em } x) \quad (2)$$

$$y = \sum_i \bar{y}_i \lambda_i \quad (\text{ligação em } y) \quad (3)$$

$$\sum_j s_j = 1 \quad (\text{exatamente um segmento ativo}) \quad (4)$$

A adjacência entre λ_i e s_j é imposta por:

$$\lambda_0 \leq s_0, \quad \lambda_i \leq s_{i-1} + s_i \quad (1 \leq i \leq K - 2), \quad \lambda_{K-1} \leq s_{K-2}.$$

Essas restrições garantem que apenas os dois λ_i adjacentes ao segmento selecionado por $s_j = 1$ possam ser positivos, reproduzindo a semântica SOS2 sem recorrer ao tipo nativo `GRB.SOS2`.

9.3. Integração com o solver

A função retorna as listas `lambdas` e `segments`, permitindo que o modelo externo referencie as variáveis auxiliares para impor restrições adicionais ou inspecionar a solução. O modelo resultante é um MILP padrão, compatível com o formato MPS e diretamente resolvível pelo motor `BranchAndBound` sem qualquer modificação, as variáveis binárias s_j participam naturalmente do processo de branching, e as variáveis λ_i contínuas são tratadas pela relaxação LP de cada nó.

10. Experimentos

Esta seção apresenta a metodologia experimental adotada para validar a corretude do solver e avaliar, de forma isolada e comparativa, o impacto dos principais componentes implementados. O objetivo é analisar o efeito do presolve, das estratégias de exploração da árvore, das regras de branching, dos cortes válidos, das heurísticas primais e do suporte a restrições SOS2.

A estratégia adotada foi baseada em uma *baseline* crua fixa, correspondente a uma versão mínima do solver, e em experimentos de ablação isolada. Em cada bloco experimental, um único componente foi ativado ou substituído, mantendo-se os demais elementos da baseline inalterados. Essa escolha permite medir o efeito marginal de cada módulo sem confundir sua contribuição com a de outros mecanismos do *Branch-and-Cut*. Ao final, também é apresentada uma comparação entre a baseline crua e uma configuração completa, composta pela melhor escolha observada para cada componente.

10.1. Objetivos experimentais

Os experimentos foram planejados com os seguintes objetivos:

- validar o funcionamento básico do solver em instâncias pequenas e médias, incluindo casos viáveis e inviáveis;
- avaliar o efeito do presolve sobre o modelo original, observando aperto de bounds, simplificação de restrições e detecção de inviabilidade;

- comparar estratégias de exploração da árvore, em especial *Depth-First Search* e *Best-First Search*;
- comparar regras de seleção de variável para branching, em particular *Strong Branching* e *Pseudocost Branching*;
- medir o impacto dos cortes GMI, MIR e Cover sobre a relaxação linear e sobre o desempenho global da busca;
- avaliar a contribuição das heurísticas primais na obtenção de incumbentes;
- demonstrar o funcionamento do suporte a SOS2 em um modelo com linearização *piecewise*;
- comparar o desempenho de uma versão mínima do solver com uma versão final mais completa.

10.2. Ambiente experimental

Todos os experimentos foram executados em um Macbook Air com processador M4 e 16 GB de memória RAM utilizando Python 3.11.6 e Gurobi 13.0.1 como *solver* das relaxações lineares. Todas as execuções foram realizadas sob o mesmo ambiente computacional, de modo que as diferenças observadas entre configurações reflitam essencialmente o comportamento algorítmico dos componentes avaliados.

10.3. Configuração-base e metodologia de comparação

Para a avaliação experimental, foi definida uma *baseline*, isto é, uma configuração mínima do solver, usada como referência comum para os testes. Essa *baseline* corresponde a uma versão simples do método, sem mecanismos adicionais de aceleração ou fortalecimento do modelo.

Salvo indicação em contrário, a *baseline* é composta por:

- presolve desativado;
- cortes desativados;
- heurísticas primais desativadas;
- estratégia de exploração da árvore fixa em BFS;

- regra de branching fixa em FirstFractional.

O impacto de cada módulo é medido de forma isolada. Por exemplo, para avaliar o presolve, compara-se a *baseline* com a *baseline* crua acrescida do presolve; para avaliar cortes, compara-se a *baseline* com a *baseline* acrescida de cortes; para avaliar heurísticas, compara-se a *baseline* com a *baseline* acrescida da heurística considerada.

Nos casos em que existem duas alternativas do mesmo módulo, como DFS versus Best-First e Strong Branching versus Pseudocost Branching, as variantes também são comparadas diretamente entre si sob as mesmas condições experimentais.

10.4. Instâncias testadas

Os experimentos foram realizados sobre um conjunto fixo de 17 instâncias.

A Tabela 1 resume tais instâncias, indicando suas principais características.

Table 1: Conjunto de instâncias utilizado nos experimentos.

Instância	n vars	m restr.	Binárias	Inteiras	Contínuas
inst_01	8	7	3	3	2
inst_02	9	8	3	4	2
inst_03	10	10	4	4	2
inst_04	12	11	4	5	3
inst_05	14	13	5	6	3
inst_06	15	14	5	6	4
inst_07	16	15	5	7	4
inst_08	17	17	6	7	4
inst_09	18	18	6	8	4
inst_10	20	19	7	8	5
inst_11	23	20	8	10	5
inst_12	24	21	8	11	5
inst_13	27	23	9	12	6
inst_14	29	25	10	13	6
inst_15	31	27	10	14	7
inst_16	36	32	12	16	8
inst_17	40	34	14	18	8

10.5. Métricas coletadas

Para cada execução, foram registradas métricas finais da solução, estatísticas da árvore de Branch-and-Bound e indicadores específicos dos módulos auxiliares do solver. As principais grandezas analisadas foram:

- status final da execução;
- valor da solução inteira final;
- valor da relaxação linear na raiz;
- valor da relaxação linear da raiz após a aplicação de cortes;
- gap da relaxação da raiz em relação ao ótimo final;
- gap da relaxação da raiz após cortes em relação ao ótimo final;
- tempo total de execução;
- tempo gasto na resolução de subproblemas LP;
- tempo gasto em heurísticas;
- tempo gasto na geração de cortes;
- número de nós explorados;
- número de nós podados;
- número de podas por inviabilidade;
- número de podas por bound;
- profundidade máxima da árvore;
- número máximo de nós abertos simultaneamente;
- número de atualizações de incumbente ao longo da execução.

Além das métricas globais, também foram coletadas estatísticas específicas por componente.

A partir dessas informações, foram construídas tabelas de resultado com foco em: qualidade da relaxação linear, esforço da árvore, distribuição de tempo entre os componentes do solver, contribuição de heurísticas e cortes, além do efeito agregado das regras de presolve.

10.6. Validação básica

O primeiro conjunto de resultados usando a *baseline* (Seção 10.3) é apresentado nas Tabelas 2, 3, 4. Para isso, o solver foi executado sobre as 17 instâncias consideradas, registrando-se as diferentes métricas. Essas tabelas serão usadas como base comparativa ao longo dos próximos testes.

Table 2: Resultado final e qualidade da relaxação - *baseline*.

instance	status	obj_value	root_lp	root_lp_after_cuts	gap_root	gap_root_after_cuts
01	optimal	27.0000	25.5000	—	0.0556	—
02	optimal	34.0000	33.1250	—	0.0257	—
03	optimal	45.0000	40.8000	—	0.0933	—
04	optimal	54.0000	50.0000	—	0.0741	—
05	optimal	60.0000	57.0833	—	0.0486	—
06	optimal	77.0000	77.0000	—	0.0000	—
07	optimal	84.0000	80.9000	—	0.0369	—
08	optimal	84.0000	80.1667	—	0.0456	—
09	optimal	98.0000	92.8333	—	0.0527	—
10	optimal	96.0000	92.6667	—	0.0347	—
12	optimal	139.0000	134.9479	—	0.0292	—
13	optimal	164.0000	156.0583	—	0.0484	—
14	optimal	182.0000	173.4487	—	0.0470	—
15	optimal	212.0000	201.0732	—	0.0515	—
16	optimal	234.0000	228.5588	—	0.0233	—
17	optimal	222.0000	214.0000	—	0.0360	—

Table 3: Tamanho e esforço da árvore Branch-and-Bound - *baseline*.

instance	nodes_explored	nodes_pruned	pruned_by_infeasibility	pruned_by_bound	max_depth	max_open_nodes
01	9	3	0	3	3	3
02	5	2	0	2	2	2
03	17	8	2	6	4	5
04	37	17	3	14	6	13
05	11	5	0	5	4	4
06	1	0	0	0	0	0
07	61	27	0	27	7	15
08	47	22	1	21	5	17
09	113	56	5	51	10	35
10	69	33	0	33	6	32
12	33	16	0	16	6	10
13	1191	593	29	564	14	392
14	1115	555	21	534	12	344
15	2883	1436	29	1407	16	888
16	973	485	56	429	11	431
17	807	399	6	393	13	247

Table 4: Tempos de execução por componente - *baseline*.

instance	runtime_total	runtime_lp	runtime_heuristics	runtime_cuts
01	0.0052	0.0041	0	0
02	0.0006	0.0003	0	0
03	0.0016	0.0007	0	0
04	0.0037	0.0015	0	0
05	0.0011	0.0005	0	0
06	0.0002	0.0001	0	0
07	0.0076	0.0033	0	0
08	0.0054	0.0023	0	0
09	0.0134	0.0059	0	0
10	0.0086	0.0039	0	0
12	0.0044	0.0020	0	0
13	0.1862	0.0858	0	0
14	0.1817	0.0824	0	0
15	0.4932	0.2197	0	0
16	0.1513	0.0621	0	0
17	0.1536	0.0661	0	0

10.7. Avaliação do *presolve*

Para avaliar o impacto do *presolve*, foram ativadas todas as rotinas implementadas: *Bound Tightening*, *Row Feasibility and Redundancy*, *Substitute Fixed Variables*, *Implicit Free Variable Detection*, *Singleton Row Reduction*, *Euclidean Reduction*, *Coefficient Strengthening*, *Chvátal-Gomory Strengthening* e *Parallel Rows*.

O objetivo dessa etapa é verificar se tais regras conseguem melhorar a formulação antes da abertura da árvore de Branch-and-Bound, por meio de aperto de *bounds*, remoção de restrições e fortalecimento algébrico do modelo. Em todas as instâncias, as nove regras foram executadas, mas o efeito prático foi heterogêneo. Nas instâncias menores, as alterações foram limitadas, com poucos *bounds* apertados e quase nenhuma remoção de linhas. Já nas instâncias maiores, especialmente 12 a 17, o *presolve* produziu efeitos mais visíveis, com destaque para a instância 16, na qual houve 16 apertos de *bounds*, remoção de 6 linhas e alteração de 10 coeficientes.

Em termos de simplificação estrutural, o *presolve* mostrou baixo custo computacional e remoção moderada de restrições, sem fixação de variáveis em nenhuma instância. Isso sugere que seu principal efeito esteve mais no for-

talecimento da formulação do que em uma redução combinatória expressiva do modelo. Ainda assim, todas as instâncias foram resolvidas até a otimalidade, e os *gaps* na raiz permaneceram relativamente baixos, chegando a zero na instância 06.

Na comparação com a configuração *baseline*, observa-se que o impacto sobre a árvore foi pequeno em várias instâncias, mas relevante em alguns casos mais difíceis. A instância 09 reduziu de 113 para 97 nós explorados, e a 13 de 1191 para 1021. O caso mais expressivo foi a instância 16, em que o número de nós caiu de 973 para 141, a profundidade máxima foi reduzida de 11 para 9 e o número máximo de nós abertos passou de 431 para 64. Houve ainda ganhos menores nas instâncias 15 e 17, enquanto em diversos casos a árvore permaneceu inalterada.

Os tempos de execução confirmam esse comportamento. Em geral, os ganhos foram modestos, mas consistentes, com pequenas reduções no tempo total e no tempo gasto nas relaxações lineares. Nas instâncias mais difíceis, o efeito foi mais claro, isto é, a instância 13 reduziu o tempo total de 0.1862 para 0.1523, a 15 de 0.4932 para 0.4577 e a 17 de 0.1536 para 0.1400. Novamente, o maior destaque foi a instância 16, cujo tempo total caiu de 0.1513 para 0.0217.

De forma geral, os resultados mostram que o *presolve* implementado é barato e pode trazer ganhos relevantes, embora de forma não uniforme. Quando as simplificações obtidas foram modestas, o efeito sobre a árvore também foi pequeno; por outro lado, quando o modelo foi mais fortemente modificado, o impacto no esforço do Branch-and-Bound foi significativo.

Table 5: Resumo do presolve por instância com *presolve*.

instance	Qtde. de Regras	Bounds Tightened	Rows Removed	Vars Fixed	Coefficients Changed	Presolve Runtime	Rows Removed (%)
01	9	2	0	0	0	0.0025	0.00
02	9	1	0	0	0	0.0008	0.00
03	9	1	0	0	1	0.0012	0.00
04	9	0	0	0	0	0.0009	0.00
05	9	1	0	0	0	0.0013	0.00
06	9	2	2	0	0	0.0013	14.28
07	9	2	1	0	0	0.0014	6.66
08	9	1	1	0	0	0.0017	5.88
09	9	2	2	0	0	0.0016	11.11
10	9	1	2	0	0	0.0019	10.52
12	9	11	1	0	0	0.0023	4.76
13	9	12	2	0	0	0.0025	8.69
14	9	13	3	0	0	0.0028	12.00
15	9	14	3	0	0	0.0031	11.11
16	9	16	6	0	10	0.0042	18.75
17	9	18	5	0	0	0.0043	14.70

Table 6: Resultado final e qualidade da relaxação com *presolve*.

instance	status	obj_value	root_lp	root_lp_after_cuts	gap_root	gap_root_after_cuts
01	optimal	27.0000	25.5000	—	0.0556	—
02	optimal	34.0000	33.1250	—	0.0257	—
03	optimal	45.0000	40.7391	—	0.0947	—
04	optimal	54.0000	50.0000	—	0.0741	—
05	optimal	60.0000	57.0833	—	0.0486	—
06	optimal	77.0000	77.0000	—	0.0000	—
07	optimal	84.0000	80.9000	—	0.0369	—
08	optimal	84.0000	80.1667	—	0.0456	—
09	optimal	98.0000	92.8333	—	0.0527	—
10	optimal	96.0000	92.6667	—	0.0347	—
12	optimal	139.0000	134.9479	—	0.0292	—
13	optimal	164.0000	156.0583	—	0.0484	—
14	optimal	182.0000	173.4487	—	0.0470	—
15	optimal	212.0000	201.0732	—	0.0515	—
16	optimal	230.0000	226.4759	—	0.0153	—
17	optimal	222.0000	214.0000	—	0.0360	—

Table 7: Tamanho e esforço da árvore Branch-and-Bound para a configuração *baseline* e com *presolve*.

instance	configuração	nodes_explored	nodes_pruned	pruned_by_infeasibility	pruned_by_bound	max_depth	max_open_nodes
01	baseline	9	3	0	3	3	3
	com presolve	9	3	0	3	3	3
02	baseline	5	2	0	2	2	2
	com presolve	5	2	0	2	2	2
03	baseline	17	8	2	6	4	5
	com presolve	17	8	1	7	5	5
04	baseline	37	17	3	14	6	13
	com presolve	37	17	3	14	6	13
05	baseline	11	5	0	5	4	4
	com presolve	11	5	0	5	4	4
06	baseline	1	0	0	0	0	0
	com presolve	1	0	0	0	0	0
07	baseline	61	27	0	27	7	15
	com presolve	61	27	0	27	7	15
08	baseline	47	22	1	21	5	17
	com presolve	47	22	1	21	5	17
09	baseline	113	56	5	51	10	35
	com presolve	97	48	4	44	10	22
10	baseline	69	33	0	33	6	32
	com presolve	69	33	0	33	6	32
12	baseline	33	16	0	16	6	10
	com presolve	33	16	0	16	6	10
13	baseline	1191	593	29	564	14	392
	com presolve	1021	509	25	484	15	273
14	baseline	1115	555	21	534	12	344
	com presolve	1111	553	20	533	12	342
15	baseline	2883	1436	29	1407	16	888
	com presolve	2833	1411	33	1378	16	870
16	baseline	973	485	56	429	11	431
	com presolve	141	68	0	68	9	64
17	baseline	807	399	6	393	13	247
	com presolve	781	386	4	382	13	247

Table 8: Tempos de execução por componente para a configuração *baseline* e com *presolve*.

instance	configuração	runtime_total	runtime_lp	runtime_heuristics	runtime_cuts
01	baseline	0.0052	0.0041	0	0
	com presolve	0.0039	0.0028	0	0
02	baseline	0.0006	0.0003	0	0
	com presolve	0.0005	0.0002	0	0
03	baseline	0.0016	0.0007	0	0
	com presolve	0.0017	0.0008	0	0
04	baseline	0.0037	0.0015	0	0
	com presolve	0.0039	0.0017	0	0
05	baseline	0.0011	0.0005	0	0
	com presolve	0.0012	0.0005	0	0
06	baseline	0.0002	0.0001	0	0
	com presolve	0.0002	0.0001	0	0
07	baseline	0.0076	0.0033	0	0
	com presolve	0.0070	0.0030	0	0
08	baseline	0.0054	0.0023	0	0
	com presolve	0.0053	0.0023	0	0
09	baseline	0.0134	0.0059	0	0
	com presolve	0.0117	0.0050	0	0
10	baseline	0.0086	0.0039	0	0
	com presolve	0.0085	0.0038	0	0
12	baseline	0.0044	0.0020	0	0
	com presolve	0.0043	0.0018	0	0
13	baseline	0.1862	0.0858	0	0
	com presolve	0.1523	0.0670	0	0
14	baseline	0.1817	0.0824	0	0
	com presolve	0.1720	0.0744	0	0
15	baseline	0.4932	0.2197	0	0
	com presolve	0.4577	0.1980	0	0
16	baseline	0.1513	0.0621	0	0
	com presolve	0.0217	0.0087	0	0
17	baseline	0.1536	0.0661	0	0
	com presolve	0.1400	0.0587	0	0

10.8. Comparação entre estratégias de exploração da árvore

A comparação entre estratégias de exploração da árvore foi conduzida mantendo-se a configuração *baseline* fixa e alterando apenas o critério de seleção do próximo nó. Assim, nesta análise, o *presolve* permanece desativado. Foram comparadas as estratégias *Breadth-First Search* (configuração de referência), *Depth-First Search* e *Best-First Search*.

Como esperado, a estratégia de exploração não altera a formulação nem a relaxação linear na raiz. Por isso, os valores de *root LP* e de *gap* na

raiz permanecem essencialmente os mesmos entre as configurações, e todas as instâncias foram resolvidas até a otimalidade. As diferenças observadas decorrem, portanto, exclusivamente da ordem em que os nós são processados e do efeito dessa ordem sobre a obtenção de incumbentes e sobre o padrão de poda ao longo da árvore.

Os resultados mostram que *breadth-first* tende a manter um número muito maior de nós abertos simultaneamente, o que era esperado pelo caráter mais “horizontal” da busca. Em contrapartida, tanto *depth-first* quanto *best-first* reduzem de forma expressiva o valor de *max_open_nodes* em praticamente todas as instâncias, especialmente nas maiores. Esse efeito é bastante claro nas instâncias 13 a 17, em que a estratégia *breadth-first* chega a manter centenas de nós abertos, enquanto *depth-first* e *best-first* operam com fronteiras muito menores.

Em termos de número de nós explorados, não há um vencedor absoluto, mas o padrão geral favorece *depth-first* e, em vários casos, também *best-first*, em comparação com a estratégia *breadth-first*. A estratégia *depth-first* apresentou reduções importantes em instâncias como 10, 13, 14, 15, 16 e 17. Na instância 15, por exemplo, o número de nós explorados caiu de 2883 para 1399, enquanto na instância 17 a redução foi de 807 para 337. A estratégia *best-first* também obteve ganhos relevantes, especialmente nas instâncias 07, 14, 15, 16 e 17, embora em algumas instâncias pequenas ou médias tenha explorado mais nós que as demais, como ocorreu em 01, 03 e 12.

A estratégia *depth-first* também mostrou um comportamento típico de aprofundamento: em várias instâncias, a profundidade máxima aumentou em relação a *breadth-first*. Já *best-first* apresentou um comportamento intermediário: em geral, manteve menos nós abertos do que *breadth-first*, mas mais do que *depth-first*, ao mesmo tempo em que conseguiu reduzir o número de nós explorados em algumas instâncias difíceis. Isso sugere que a priorização por qualidade do nó pode ser benéfica, embora seu efeito dependa mais fortemente da estrutura específica de cada instância.

Os tempos de execução confirmam essa tendência. Nas instâncias pequenas, as diferenças são pouco expressivas, já que o esforço total é muito baixo independentemente da estratégia. Entretanto, nas instâncias maiores, *depth-first* e *best-first* superaram de forma consistente a estratégia *breadth-first*. A estratégia *depth-first* foi a melhor em instâncias como 13, 15, 16 e 17, enquanto *best-first* obteve o menor tempo em casos como 07 e 14. Em particular, na instância 15 o tempo total caiu de 0.4932 com *breadth-first* para 0.2300 com *depth-first*, e na instância 17 de 0.1536 para 0.0598. Esses ganhos

acompanham a redução do número de nós explorados e, principalmente, a forte diminuição do número de nós abertos simultaneamente.

De forma geral, os resultados indicam que a estratégia *breadth-first*, embora simples, foi a menos eficiente no conjunto testado, sobretudo nas instâncias maiores. A estratégia *depth-first* apresentou o comportamento mais robusto, com reduções frequentes no tamanho da árvore e no tempo total, além de exigir muito menos memória implícita para manter a fronteira de busca. A estratégia *best-first* também se mostrou competitiva e, em alguns casos, superior às demais, mas com desempenho menos uniforme. Assim, para este conjunto de instâncias, *depth-first* aparece como a alternativa mais consistente à configuração *baseline*.

Table 9: Resultado final e qualidade da relaxação - *Depth-First Search*.

instance	status	obj_value	root_lp	root_lp_after_cuts	gap_root	gap_root_after_cuts
01	optimal	27.0000	25.5000	—	0.0556	—
02	optimal	34.0000	33.1250	—	0.0257	—
03	optimal	45.0000	40.8000	—	0.0933	—
04	optimal	54.0000	50.0000	—	0.0741	—
05	optimal	60.0000	57.0833	—	0.0486	—
06	optimal	77.0000	77.0000	—	0.0000	—
07	optimal	84.0000	80.9000	—	0.0369	—
08	optimal	84.0000	80.1667	—	0.0456	—
09	optimal	98.0000	92.8333	—	0.0527	—
10	optimal	96.0000	92.6667	—	0.0347	—
12	optimal	139.0000	134.9479	—	0.0292	—
13	optimal	164.0000	156.0583	—	0.0484	—
14	optimal	182.0000	173.4487	—	0.0470	—
15	optimal	212.0000	201.0732	—	0.0515	—
16	optimal	234.0000	228.5588	—	0.0233	—
17	optimal	222.0000	214.0000	—	0.0360	—

Table 10: Tamanho e esforço da árvore Branch-and-Bound para as estratégias *breadth-first*, *depth-first* e *best-first*.

instance	estratégia	nodes_explored	nodes_pruned	pruned_by_infeasibility	pruned_by_bound	max_depth	max_open_nodes
01	breadth-first	9	3	0	3	3	3
	depth-first	9	3	0	3	3	2
	best-first	13	4	0	4	4	4
02	breadth-first	5	2	0	2	2	2
	depth-first	17	6	0	6	6	5
	best-first	3	1	0	1	1	1
03	breadth-first	17	8	2	6	4	5
	depth-first	15	7	2	5	4	2
	best-first	39	17	8	9	7	8
04	breadth-first	37	17	3	14	6	13
	depth-first	29	13	1	12	6	4
	best-first	35	16	4	12	6	8
05	breadth-first	11	5	0	5	4	4
	depth-first	11	5	0	5	4	3
	best-first	15	7	0	7	4	6
06	breadth-first	1	0	0	0	0	0
	depth-first	1	0	0	0	0	0
	best-first	1	0	0	0	0	0
07	breadth-first	61	27	0	27	7	15
	depth-first	81	37	0	37	8	5
	best-first	43	19	0	19	7	6
08	breadth-first	47	22	1	21	5	17
	depth-first	47	20	1	19	5	5
	best-first	45	20	0	20	6	9
09	breadth-first	113	56	5	51	10	35
	depth-first	105	50	8	42	10	6
	best-first	141	68	13	55	10	12
10	breadth-first	69	33	0	33	6	32
	depth-first	55	23	1	22	8	7
	best-first	69	30	1	29	8	14
12	breadth-first	33	16	0	16	6	10
	depth-first	39	18	0	18	6	5
	best-first	47	22	0	22	7	10
13	breadth-first	1191	593	29	564	14	392
	depth-first	899	441	30	411	16	12
	best-first	959	470	29	441	16	22
14	breadth-first	1115	555	21	534	12	344
	depth-first	613	303	10	293	18	14
	best-first	533	264	3	261	12	19
15	breadth-first	2883	1436	29	1407	16	888
	depth-first	1399	693	9	684	17	14
	best-first	1687	837	17	820	17	26
16	breadth-first	973	485	56	429	11	431
	depth-first	681	335	190	145	24	15
	best-first	745	367	195	172	22	30
17	breadth-first	807	399	6	393	13	247
	depth-first	337	165	6	159	14	10
	best-first	527	260	17	243	16	21

Table 11: Tempos de execução por componente para as estratégias *breadth-first*, *depth-first* e *best-first*.

instance	estratégia	runtime_total	runtime_lp	runtime_heuristics	runtime_cuts
01	breadth-first	0.0052	0.0041	0	0
	depth-first	0.0049	0.0036	0	0
	best-first	0.0055	0.0044	0	0
02	breadth-first	0.0006	0.0003	0	0
	depth-first	0.0018	0.0008	0	0
	best-first	0.0003	0.0001	0	0
03	breadth-first	0.0016	0.0007	0	0
	depth-first	0.0016	0.0007	0	0
	best-first	0.0034	0.0014	0	0
04	breadth-first	0.0037	0.0015	0	0
	depth-first	0.0030	0.0013	0	0
	best-first	0.0039	0.0017	0	0
05	breadth-first	0.0011	0.0005	0	0
	depth-first	0.0014	0.0005	0	0
	best-first	0.0017	0.0007	0	0
06	breadth-first	0.0002	0.0001	0	0
	depth-first	0.0002	0.0001	0	0
	best-first	0.0002	0.0001	0	0
07	breadth-first	0.0076	0.0033	0	0
	depth-first	0.0089	0.0038	0	0
	best-first	0.0048	0.0021	0	0
08	breadth-first	0.0054	0.0023	0	0
	depth-first	0.0053	0.0023	0	0
	best-first	0.0051	0.0022	0	0
09	breadth-first	0.0134	0.0059	0	0
	depth-first	0.0128	0.0054	0	0
	best-first	0.0171	0.0073	0	0
10	breadth-first	0.0086	0.0039	0	0
	depth-first	0.0067	0.0028	0	0
	best-first	0.0086	0.0036	0	0
12	breadth-first	0.0044	0.0020	0	0
	depth-first	0.0055	0.0024	0	0
	best-first	0.0065	0.0029	0	0
13	breadth-first	0.1862	0.0858	0	0
	depth-first	0.1355	0.0595	0	0
	best-first	0.1481	0.0642	0	0
14	breadth-first	0.1817	0.0824	0	0
	depth-first	0.0971	0.0425	0	0
	best-first	0.0844	0.0370	0	0
15	breadth-first	0.4932	0.2197	0	0
	depth-first	0.2300	0.0992	0	0
	best-first	0.2837	0.1221	0	0
16	breadth-first	0.1513	0.0621	0	0
	depth-first	0.0970	0.0370	0	0
	best-first	0.1077	0.0411	0	0
17	breadth-first	0.1536	0.0661	0	0
	depth-first	0.0598	0.0244	0	0
	best-first	0.0948	0.0382	0	0

10.9. Comparação entre regras de branching

Para avaliar a seleção da variável de branching, foram comparadas as regras *Strong Branching* e *Pseudocost Branching*, mantendo-se inalterados os demais componentes da configuração *baseline*.

Como a regra de branching atua apenas na escolha da variável a ser ramificada, ela não altera a relaxação linear na raiz. Assim, os valores de *root LP* e do *gap* inicial permanecem os mesmos, e todas as instâncias foram resolvidas até a otimalidade. As diferenças observadas decorrem, portanto, exclusivamente da qualidade das decisões de ramificação ao longo da árvore.

Os resultados mostram que ambas as regras superaram a *baseline* em diversas instâncias, especialmente nas de maior porte. Em geral, tanto *strong branching* quanto *pseudocost* reduziram de forma significativa o número de nós explorados, a profundidade da árvore e o número máximo de nós abertos. Esse efeito foi particularmente forte nas instâncias 13, 14, 15, 16 e 17. Na instância 15, por exemplo, o número de nós explorados caiu de 2883 na *baseline* para 355 com *strong branching* e para 431 com *pseudocost*. Na instância 17, a redução foi ainda mais expressiva: de 807 nós para 69 com *strong branching* e 101 com *pseudocost*. Em termos de tamanho de árvore, portanto, as duas estratégias mostraram-se superiores à escolha padrão.

Na comparação entre elas, o *strong branching* produziu, em geral, as árvores menores. Isso aparece de forma nítida nas instâncias 09, 13, 14, 15 e 17, em que essa regra explorou menos nós do que *pseudocost*. Em contrapartida, a vantagem estrutural do *strong branching* nem sempre se converteu em melhor tempo total. Como essa regra exige a avaliação explícita de candidatos antes da decisão de branching, seu custo por nó é mais alto, o que aumenta o tempo gasto na resolução de relaxações lineares.

Esse trade-off fica evidente nos tempos de execução. Em instâncias pequenas e médias, tanto *strong branching* quanto *pseudocost* frequentemente aumentaram o tempo total em relação à *baseline*, mesmo quando reduziram a árvore, pois o overhead da escolha de branching superou o ganho obtido. Já nas instâncias maiores, o *pseudocost* apresentou o comportamento mais favorável. Na instância 15, por exemplo, o tempo total caiu de 0.4932 na *baseline* para 0.1150 com *pseudocost*, enquanto com *strong branching* caiu para 0.2733. Na instância 16, o contraste é ainda mais forte: embora o *strong branching* tenha reduzido a árvore em relação à *baseline*, seu tempo total subiu para 0.2831, ao passo que o *pseudocost* reduziu o tempo para apenas 0.0396. Na instância 17, ambas as regras foram vantajosas, mas *pseudocost* novamente apresentou o menor tempo.

De forma geral, os resultados indicam que o *strong branching* é mais eficaz para reduzir o tamanho da árvore, produzindo decisões de ramificação de maior qualidade. No entanto, esse ganho vem acompanhado de um custo computacional elevado por decisão. Já o *pseudocost branching* apresentou o melhor equilíbrio entre qualidade da árvore e custo de processamento, sendo a alternativa mais vantajosa no conjunto testado quando o critério de comparação é tempo total de execução.

Table 12: Tamanho e esforço da árvore Branch-and-Bound para diferentes regras de branching.

instance	regra de branching	nodes_explored	nodes_pruned	pruned_by_infeasibility	pruned_by_bound	max_depth	max_open_nodes
01	baseline	9	3	0	3	3	3
	strong branching	11	4	0	4	3	4
	pseudocost	11	4	0	4	3	4
02	baseline	5	2	0	2	2	2
	strong branching	5	1	0	1	2	2
	pseudocost	5	1	0	1	2	2
03	baseline	17	8	2	6	4	5
	strong branching	13	6	3	3	4	4
	pseudocost	13	6	3	3	4	4
04	baseline	37	17	3	14	6	13
	strong branching	25	11	4	7	5	7
	pseudocost	25	11	4	7	5	7
05	baseline	11	5	0	5	4	4
	strong branching	13	6	1	5	4	5
	pseudocost	13	6	1	5	4	5
06	baseline	1	0	0	0	0	0
	strong branching	1	0	0	0	0	0
	pseudocost	1	0	0	0	0	0
07	baseline	61	27	0	27	7	15
	strong branching	59	28	2	26	6	18
	pseudocost	53	24	1	23	6	13
08	baseline	47	22	1	21	5	17
	strong branching	39	18	0	18	5	16
	pseudocost	41	19	0	19	6	16
09	baseline	113	56	5	51	10	35
	strong branching	29	14	4	10	6	9
	pseudocost	35	17	4	13	6	9
10	baseline	69	33	0	33	6	32
	strong branching	37	18	1	17	5	18
	pseudocost	37	18	1	17	5	18
12	baseline	33	16	0	16	6	10
	strong branching	17	8	2	6	6	4
	pseudocost	17	8	2	6	6	4
13	baseline	1191	593	29	564	14	392
	strong branching	217	108	33	75	10	80
	pseudocost	303	151	24	127	11	106
14	baseline	1115	555	21	534	12	344
	strong branching	373	186	35	151	10	167
	pseudocost	403	201	29	172	11	175
15	baseline	2883	1436	29	1407	16	888
	strong branching	355	176	37	139	11	149
	pseudocost	431	214	28	186	12	181
16	baseline	973	485	56	429	11	431
	strong branching	391	194	31	163	9	180
	pseudocost	117	58	5	53	8	50
17	baseline	807	399	6	393	13	247
	strong branching	69	34	5	29	9	20
	pseudocost	101	50	6	44	8	39

Table 13: Tempos de execução por componente para diferentes regras de branching.

instance	regra de branching	runtime_total	runtime_lp	runtime_heuristics	runtime_cuts
01	baseline	0.0052	0.0041	0	0
	strong branching	0.0083	0.0068	0	0
	pseudocost	0.0062	0.0045	0	0
02	baseline	0.0006	0.0003	0	0
	strong branching	0.0013	0.0007	0	0
	pseudocost	0.0013	0.0006	0	0
03	baseline	0.0016	0.0007	0	0
	strong branching	0.0037	0.0020	0	0
	pseudocost	0.0037	0.0018	0	0
04	baseline	0.0037	0.0015	0	0
	strong branching	0.0079	0.0040	0	0
	pseudocost	0.0082	0.0041	0	0
05	baseline	0.0011	0.0005	0	0
	strong branching	0.0033	0.0016	0	0
	pseudocost	0.0033	0.0015	0	0
06	baseline	0.0002	0.0001	0	0
	strong branching	0.0002	0.0001	0	0
	pseudocost	0.0002	0.0001	0	0
07	baseline	0.0076	0.0033	0	0
	strong branching	0.0164	0.0085	0	0
	pseudocost	0.0124	0.0062	0	0
08	baseline	0.0054	0.0023	0	0
	strong branching	0.0138	0.0074	0	0
	pseudocost	0.0129	0.0067	0	0
09	baseline	0.0134	0.0059	0	0
	strong branching	0.0118	0.0065	0	0
	pseudocost	0.0112	0.0060	0	0
10	baseline	0.0086	0.0039	0	0
	strong branching	0.0134	0.0071	0	0
	pseudocost	0.0134	0.0071	0	0
12	baseline	0.0044	0.0020	0	0
	strong branching	0.0112	0.0064	0	0
	pseudocost	0.0097	0.0055	0	0
13	baseline	0.1862	0.0858	0	0
	strong branching	0.1358	0.0781	0	0
	pseudocost	0.0825	0.0427	0	0
14	baseline	0.1817	0.0824	0	0
	strong branching	0.2103	0.1178	0	0
	pseudocost	0.0987	0.0492	0	0
15	baseline	0.4932	0.2197	0	0
	strong branching	0.2733	0.1576	0	0
	pseudocost	0.1150	0.0581	0	0
16	baseline	0.1513	0.0621	0	0
	strong branching	0.2831	0.1506	0	0
	pseudocost	0.0396	0.0198	0	0
17	baseline	0.1536	0.0661	0	0
	strong branching	0.0548	0.0315	0	0
	pseudocost	0.0470	0.0254	0	0

10.10. Avaliação dos cortes

A avaliação dos cortes foi realizada mantendo-se a configuração *baseline* fixa e ativando os mecanismos de separação. Em cada instância, foram considerados em conjunto os quatro cortes implementados: *Mixed Integer Gomory Cut*, *MIR*, *Lifting & Extending* e *Cover Cut*.

Diferentemente do *presolve* e das estratégias de exploração da árvore, os cortes atuam diretamente na qualidade da relaxação linear. Os resultados mostram que, em praticamente todas as instâncias, houve melhora do valor da relaxação na raiz e redução do *gap* inicial. Em alguns casos, esse efeito foi particularmente forte: nas instâncias 02 e 04, por exemplo, o *gap* na raiz foi fechado completamente após a separação, eliminando a necessidade de uma árvore de busca significativa. Também se observam reduções expressivas em instâncias como 10, 12 e 17, indicando que os cortes tornaram a formulação substancialmente mais forte logo no nó raiz.

Esse fortalecimento refletiu-se diretamente no tamanho da árvore de Branch-and-Bound. Em várias instâncias houve redução clara no número de nós explorados, na profundidade máxima e no número de nós abertos. Os casos mais marcantes são as instâncias 10, 15, 16 e 17. Na instância 10, o número de nós caiu de 69 para 5; na instância 15, de 2883 para 653; na instância 16, de 973 para 139; e na instância 17, de 807 para 175. Em problemas pequenos, como 02 e 04, a melhora na relaxação foi suficiente para praticamente eliminar a árvore. Houve, no entanto, alguns casos em que o efeito estrutural foi modesto, como nas instâncias 03, 07 e 09.

Os tempos de execução mostram que o impacto dos cortes depende do equilíbrio entre o custo de separação e a redução do esforço da árvore. Em instâncias pequenas, esse custo adicional frequentemente superou os benefícios, levando a aumentos no tempo total, como ocorreu em 01, 02, 05, 07, 08 e 12. A instância 03 é o caso mais extremo: embora a árvore não tenha melhorado, o tempo total aumentou fortemente devido ao custo de separação, tornando os cortes desfavoráveis nesse caso. Por outro lado, nas instâncias em que o fortalecimento da relaxação foi suficiente para reduzir drasticamente a árvore, os ganhos foram expressivos. Isso ocorre de forma clara nas instâncias 10, 15, 16 e 17, nas quais o tempo total caiu de maneira substancial em relação à *baseline*.

De forma geral, os resultados indicam que os cortes foram eficazes para fortalecer a relaxação linear e reduzir o tamanho da árvore, sobretudo nas instâncias mais difíceis. No entanto, seu benefício em tempo total não foi uniforme, pois depende do custo de geração dos cortes. Assim, os cortes

se mostraram muito vantajosos quando a melhora na relaxação compensou o *overhead* da separação, mas menos atrativos em instâncias pequenas ou pouco sensíveis ao fortalecimento da formulação.

Table 14: Resultado final e qualidade da relaxação - Cortes.

instance	status	obj_value	root_lp	root_lp_after_cuts	gap_root	gap_root_after_cuts
01	optimal	27.0000	25.5000	26.2000	0.0556	0.0296
02	optimal	34.0000	33.1250	34.0000	0.0257	0.0000
03	optimal	45.0000	40.8000	42.5556	0.0933	0.0543
04	optimal	54.0000	50.0000	54.0000	0.0741	0.0000
05	optimal	60.0000	57.0833	59.0000	0.0486	0.0167
06	optimal	77.0000	77.0000	—	0.0000	—
07	optimal	84.0000	80.9000	82.8333	0.0369	0.0139
08	optimal	84.0000	80.1667	82.4000	0.0456	0.0190
09	optimal	98.0000	92.8333	94.4412	0.0527	0.0363
10	optimal	96.0000	92.6667	95.8605	0.0347	0.0015
12	optimal	139.0000	134.9479	137.6342	0.0292	0.0098
13	optimal	164.0000	156.0583	157.5845	0.0484	0.0391
14	optimal	182.0000	173.4487	176.5226	0.0470	0.0301
15	optimal	212.0000	201.0732	205.6341	0.0515	0.0300
16	optimal	234.0000	228.5588	230.8500	0.0233	0.0135
17	optimal	222.0000	214.0000	218.5714	0.0360	0.0154

Table 15: Tamanho e esforço da árvore Branch-and-Bound para a configuração *baseline* e com cortes.

instance	configuração	nodes_explored	nodes_pruned	pruned_by_infeasibility	pruned_by_bound	max_depth	max_open_nodes
01	baseline	9	3	0	3	3	3
	cortes	7	2	0	2	2	3
02	baseline	5	2	0	2	2	2
	cortes	1	0	0	0	0	0
03	baseline	17	8	2	6	4	5
	cortes	17	8	2	6	5	5
04	baseline	37	17	3	14	6	13
	cortes	5	2	0	2	2	2
05	baseline	11	5	0	5	4	4
	cortes	5	2	0	2	2	1
06	baseline	1	0	0	0	0	0
	cortes	1	0	0	0	0	0
07	baseline	61	27	0	27	7	15
	cortes	47	21	0	21	7	16
08	baseline	47	22	1	21	5	17
	cortes	41	19	0	19	5	16
09	baseline	113	56	5	51	10	35
	cortes	105	50	4	46	9	26
10	baseline	69	33	0	33	6	32
	cortes	5	2	0	2	2	2
12	baseline	33	16	0	16	6	10
	cortes	13	6	0	6	3	5
13	baseline	1191	593	29	564	14	392
	cortes	965	480	30	450	13	258
14	baseline	1115	555	21	534	12	344
	cortes	687	341	8	333	12	267
15	baseline	2883	1436	29	1407	16	888
	cortes	653	323	5	318	14	149
16	baseline	973	485	56	429	11	431
	cortes	139	68	0	68	10	34
17	baseline	807	399	6	393	13	247
	cortes	175	85	1	84	8	83

Table 16: Tempos de execução por componente para a configuração *baseline* e com cortes.

instance	configuração	runtime_total	runtime_lp	runtime_heuristics	runtime_cuts
01	baseline	0.0052	0.0041	0	0
	cortes	0.0134	0.0055	0	0.0068
02	baseline	0.0006	0.0003	0	0
	cortes	0.0014	0.0001	0	0.0012
03	baseline	0.0016	0.0007	0	0
	cortes	0.3577	0.0013	0	0.3550
04	baseline	0.0037	0.0015	0	0
	cortes	0.0030	0.0003	0	0.0022
05	baseline	0.0011	0.0005	0	0
	cortes	0.0026	0.0003	0	0.0019
06	baseline	0.0002	0.0001	0	0
	cortes	0.0011	0.0001	0	0.0009
07	baseline	0.0076	0.0033	0	0
	cortes	0.0093	0.0035	0	0.0019
08	baseline	0.0054	0.0023	0	0
	cortes	0.0113	0.0034	0	0.0041
09	baseline	0.0134	0.0059	0	0
	cortes	0.0212	0.0085	0	0.0043
10	baseline	0.0086	0.0039	0	0
	cortes	0.0042	0.0006	0	0.0031
12	baseline	0.0044	0.0020	0	0
	cortes	0.0077	0.0020	0	0.0040
13	baseline	0.1862	0.0858	0	0
	cortes	0.2004	0.1038	0	0.0064
14	baseline	0.1817	0.0824	0	0
	cortes	0.1516	0.0781	0	0.0066
15	baseline	0.4932	0.2197	0	0
	cortes	0.1449	0.0724	0	0.0063
16	baseline	0.1513	0.0621	0	0
	cortes	0.0375	0.0175	0	0.0057
17	baseline	0.1536	0.0661	0	0
	cortes	0.0448	0.0204	0	0.0049

10.11. Avaliação das heurísticas primais

As heurísticas primais foram avaliadas em relação à configuração *baseline*, sendo ativadas em conjunto. O foco principal dessa análise é medir seu efeito sobre a obtenção de incumbentes e, indiretamente, sobre a poda por limite ao longo da árvore.

Os resultados mostram que as heurísticas tiveram impacto positivo na redução do tamanho da árvore em boa parte das instâncias. Em geral, a presença de incumbentes mais cedo aumentou a poda por *bound*, reduzindo o

número de nós explorados e o número máximo de nós abertos. Esse efeito foi discreto nas instâncias pequenas, mas tornou-se bastante claro nas maiores. Na instância 10, por exemplo, o número de nós caiu de 69 para 21; na 13, de 1191 para 433; na 15, de 2883 para 997; na 16, de 973 para 149; e na 17, de 807 para 213. Houve também reduções expressivas em *max_open_nodes*, como em 15, 16 e 17, sugerindo que as heurísticas ajudaram a controlar melhor a expansão da árvore.

Apesar desse efeito estrutural favorável, o impacto no tempo total foi amplamente negativo. Em todas as instâncias, o custo das heurísticas dominou o tempo de execução, superando com folga a economia obtida na resolução de relaxações lineares. Isso é particularmente visível nas instâncias maiores: na instância 13, o tempo total subiu de 0.1862 para 2.6803; na 15, de 0.4932 para 6.1676; e na 17, de 0.1536 para 1.9241. Mesmo quando houve forte redução da árvore, como nas instâncias 16 e 17, o tempo gasto pelas heurísticas foi muito superior ao ganho proporcionado na busca.

Assim, os resultados indicam que as heurísticas primais implementadas foram eficazes para encontrar soluções incumbentes capazes de intensificar a poda e reduzir o esforço estrutural do Branch-and-Bound. No entanto, no conjunto testado, esse benefício não se converteu em melhor desempenho computacional global, pois o custo de execução das heurísticas foi alto demais. Portanto, embora úteis do ponto de vista da qualidade da busca, as heurísticas se mostraram pouco vantajosas quando o critério principal é o tempo total de solução.

Table 17: Tempos de execução por componente para a configuração *baseline* e com heurísticas.

instance	configuração	runtime_total	runtime_lp	runtime_heuristics	runtime_cuts
01	baseline	0.0052	0.0041	0	0
	heurísticas	0.0289	0.0028	0.0244	0
02	baseline	0.0006	0.0003	0	0
	heurísticas	0.0087	0.0001	0.0082	0
03	baseline	0.0016	0.0007	0	0
	heurísticas	0.0475	0.0006	0.0452	0
04	baseline	0.0037	0.0015	0	0
	heurísticas	0.0921	0.0011	0.0878	0
05	baseline	0.0011	0.0005	0	0
	heurísticas	0.0506	0.0005	0.0484	0
06	baseline	0.0002	0.0001	0	0
	heurísticas	0.0006	0.0001	0.0003	0
07	baseline	0.0076	0.0033	0	0
	heurísticas	0.1613	0.0020	0.1531	0
08	baseline	0.0054	0.0023	0	0
	heurísticas	0.1555	0.0017	0.1488	0
09	baseline	0.0134	0.0059	0	0
	heurísticas	0.4316	0.0056	0.4135	0
10	baseline	0.0086	0.0039	0	0
	heurísticas	0.1295	0.0013	0.1251	0
12	baseline	0.0044	0.0020	0	0
	heurísticas	0.2260	0.0021	0.2192	0
13	baseline	0.1862	0.0858	0	0
	heurísticas	2.6803	0.0375	2.5712	0
14	baseline	0.1817	0.0824	0	0
	heurísticas	2.5637	0.0339	2.4681	0
15	baseline	0.4932	0.2197	0	0
	heurísticas	6.1676	0.0887	5.9111	0
16	baseline	0.1513	0.0621	0	0
	heurísticas	0.8764	0.0108	0.8397	0
17	baseline	0.1536	0.0661	0	0
	heurísticas	1.9241	0.0202	1.8528	0

Table 18: Tamanho e esforço da árvore Branch-and-Bound para a configuração *baseline* e com heurísticas.

instance	configuração	nodes_explored	nodes_pruned	pruned_by_infeasibility	pruned_by_bound	max_depth	max_open_nodes
01	baseline	9	3	0	3	3	3
	heurísticas	9	5	0	5	3	3
02	baseline	5	2	0	2	2	2
	heurísticas	3	2	0	2	1	1
03	baseline	17	8	2	6	4	5
	heurísticas	15	8	2	6	4	4
04	baseline	37	17	3	14	6	13
	heurísticas	25	13	0	13	6	6
05	baseline	11	5	0	5	4	4
	heurísticas	11	6	0	6	4	4
06	baseline	1	0	0	0	0	0
	heurísticas	1	1	0	1	0	0
07	baseline	61	27	0	27	7	15
	heurísticas	37	19	0	19	7	8
08	baseline	47	22	1	21	5	17
	heurísticas	31	16	0	16	5	11
09	baseline	113	56	5	51	10	35
	heurísticas	93	47	4	43	10	18
10	baseline	69	33	0	33	6	32
	heurísticas	21	11	0	11	5	7
12	baseline	33	16	0	16	6	10
	heurísticas	29	15	0	15	6	7
13	baseline	1191	593	29	564	14	392
	heurísticas	433	217	8	209	14	75
14	baseline	1115	555	21	534	12	344
	heurísticas	383	192	3	189	12	85
15	baseline	2883	1436	29	1407	16	888
	heurísticas	997	499	8	491	16	182
16	baseline	973	485	56	429	11	431
	heurísticas	149	75	4	71	11	32
17	baseline	807	399	6	393	13	247
	heurísticas	213	107	0	107	13	32

Table 19: Tempos de execução por componente - Heurísticas.

instance	runtime_total	runtime_lp	runtime_heuristics	runtime_cuts
01	0.0289	0.0028	0.0244	0
02	0.0087	0.0001	0.0082	0
03	0.0475	0.0006	0.0452	0
04	0.0921	0.0011	0.0878	0
05	0.0506	0.0005	0.0484	0
06	0.0006	0.0001	0.0003	0
07	0.1613	0.0020	0.1531	0
08	0.1555	0.0017	0.1488	0
09	0.4316	0.0056	0.4135	0
10	0.1295	0.0013	0.1251	0
12	0.2260	0.0021	0.2192	0
13	2.6803	0.0375	2.5712	0
14	2.5637	0.0339	2.4681	0
15	6.1676	0.0887	5.9111	0
16	0.8764	0.0108	0.8397	0
17	1.9241	0.0202	1.8528	0

10.12. Comparação entre a configuração *baseline* e a configuração completa

Nesta seção, compara-se a configuração *baseline*, definida como uma versão crua do solver, com uma configuração mais completa, que combina mecanismos adicionais de fortalecimento e aceleração do *Branch-and-Bound*. A configuração *baseline* utiliza:

- *presolve* desativado;
- cortes desativados;
- heurísticas primais desativadas;
- estratégia de exploração da árvore fixa em BFS;
- regra de *branching* fixa em `FirstFractional`.

Já a configuração completa combina:

- estratégia de exploração da árvore em profundidade (*Depth First*);
- regra de *branching* `PseudoCostRule`;

- todos os *presolves* implementados: *Bound Tightening*, *Row Feasibility and Redundancy*, *Substitute Fixed Variables*, *Implicit Free Variable Detection*, *Singleton Row Reduction*, *Euclidean Reduction*, *Coefficient Strengthening*, *Chvátal-Gomory Strengthening* e *Parallel Rows*;
- heurísticas primais **RoundingHeuristic** em todos os nós, **FixAndPropagate** em 30% dos nós, **RINS** em 30%, **FractionalDiving** em 30% e **FeasibilityPump** em 30%;
- todos os planos de corte implementados: **MIG**, **MIR**, *Lifting and Extending* e *Cover Cut*.

O objetivo desta comparação é avaliar, de forma integrada, o efeito combinado de uma política de busca mais informada, fortalecimento da relaxação linear, redução do modelo por *presolve* e uso de heurísticas primais na obtenção de incumbentes mais cedo.

10.12.1. Resultados comparativos

As Tabelas 20, 21 e 22 apresentam, respectivamente, a comparação da qualidade da relaxação, do esforço da árvore e dos tempos de execução entre as duas configurações.

Table 20: Resultado final e qualidade da relaxação para a configuração *baseline* e a configuração completa.

instance	configuração	status	obj_value	root_lp	root_lp_after_cuts	gap_root	gap_root_after_cuts
01	baseline	optimal	27.0000	25.5000	–	0.0556	–
	completa	optimal	27.0000	25.5000	26.2000	0.0556	0.0296
02	baseline	optimal	34.0000	33.1250	–	0.0257	–
	completa	optimal	34.0000	33.1250	34.0000	0.0257	0.0000
03	baseline	optimal	45.0000	40.8000	–	0.0933	–
	completa	optimal	45.0000	40.7391	42.5256	0.0947	0.0550
04	baseline	optimal	54.0000	50.0000	–	0.0741	–
	completa	optimal	54.0000	50.0000	54.0000	0.0741	0.0000
05	baseline	optimal	60.0000	57.0833	–	0.0486	–
	completa	optimal	60.0000	57.0833	59.3333	0.0486	0.0111
06	baseline	optimal	77.0000	77.0000	–	0.0000	–
	completa	optimal	77.0000	77.0000	–	0.0000	–
07	baseline	optimal	84.0000	80.9000	–	0.0369	–
	completa	optimal	84.0000	80.9000	82.7778	0.0369	0.0146
08	baseline	optimal	84.0000	80.1667	–	0.0456	–
	completa	optimal	84.0000	80.1667	82.4000	0.0456	0.0190
09	baseline	optimal	98.0000	92.8333	–	0.0527	–
	completa	optimal	98.0000	92.8333	94.5357	0.0527	0.0353
10	baseline	optimal	96.0000	92.6667	–	0.0347	–
	completa	optimal	96.0000	92.6667	95.8605	0.0347	0.0015
12	baseline	optimal	139.0000	134.9479	–	0.0292	–
	completa	optimal	139.0000	134.9479	137.6342	0.0292	0.0098
13	baseline	optimal	164.0000	156.0583	–	0.0484	–
	completa	optimal	164.0000	156.0583	157.5845	0.0484	0.0391
14	baseline	optimal	182.0000	173.4487	–	0.0470	–
	completa	optimal	182.0000	173.4487	176.5226	0.0301	0.0301
15	baseline	optimal	212.0000	201.0732	–	0.0515	–
	completa	optimal	212.0000	201.0732	205.6341	0.0515	0.0300
16	baseline	optimal	234.0000	228.5588	–	0.0233	–
	completa	optimal	230.0000	226.4759	228.6748	0.0153	0.0058
17	baseline	optimal	222.0000	214.0000	–	0.0360	–
	completa	optimal	222.0000	214.0000	218.5714	0.0360	0.0154

Table 21: Tamanho e esforço da árvore Branch-and-Bound para a configuração *baseline* e a configuração completa.

instance	configuração	nodes_explored	nodes_pruned	pruned_by_infeasibility	pruned_by_bound	max_depth	max_open_nodes
01	baseline	9	3	0	3	3	3
	completa	7	4	0	4	2	2
02	baseline	5	2	0	2	2	2
	completa	1	1	0	1	0	0
03	baseline	17	8	2	6	4	5
	completa	7	4	1	3	2	2
04	baseline	37	17	3	14	6	13
	completa	1	1	0	1	0	0
05	baseline	11	5	0	5	4	4
	completa	3	2	0	2	1	1
06	baseline	1	0	0	0	0	0
	completa	1	1	0	1	0	0
07	baseline	61	27	0	27	7	15
	completa	27	14	0	14	6	3
08	baseline	47	22	1	21	5	17
	completa	65	33	1	32	7	6
09	baseline	113	56	5	51	10	35
	completa	27	14	3	11	6	5
10	baseline	69	33	0	33	6	32
	completa	33	17	3	14	6	5
12	baseline	33	16	0	16	6	10
	completa	7	4	0	4	3	1
13	baseline	1191	593	29	564	14	392
	completa	253	127	13	114	13	11
14	baseline	1115	555	21	534	12	344
	completa	139	70	8	62	10	8
15	baseline	2883	1436	29	1407	16	888
	completa	103	52	3	49	8	6
16	baseline	973	485	56	429	11	431
	completa	21	11	0	11	5	4
17	baseline	807	399	6	393	13	247
	completa	35	18	1	17	7	5

Table 22: Tempos de execução por componente para a configuração *baseline* e a configuração completa.

instance	configuração	runtime_total	runtime_lp	runtime_heuristics	runtime_cuts
01	baseline	0.0052	0.0041	0	0
	completa	0.0115	0.0033	0.0042	0.0024
02	baseline	0.0006	0.0003	0	0
	completa	0.0021	0.0001	0.0005	0.0013
03	baseline	0.0016	0.0007	0	0
	completa	0.2599	0.0014	0.0024	0.2546
04	baseline	0.0037	0.0015	0	0
	completa	0.0027	0.0001	0.0003	0.0020
05	baseline	0.0011	0.0005	0	0
	completa	0.0039	0.0006	0.0006	0.0019
06	baseline	0.0002	0.0001	0	0
	completa	0.0011	0.0001	0.0001	0.0007
07	baseline	0.0076	0.0033	0	0
	completa	0.0353	0.0055	0.0228	0.0017
08	baseline	0.0054	0.0023	0	0
	completa	0.0739	0.0083	0.0515	0.0040
09	baseline	0.0134	0.0059	0	0
	completa	0.0301	0.0068	0.0149	0.0024
10	baseline	0.0086	0.0039	0	0
	completa	0.0442	0.0067	0.0281	0.0032
12	baseline	0.0044	0.0020	0	0
	completa	0.0147	0.0039	0.0045	0.0037
13	baseline	0.1862	0.0858	0	0
	completa	0.3093	0.0390	0.2273	0.0047
14	baseline	0.1817	0.0824	0	0
	completa	0.2388	0.0331	0.1742	0.0052
15	baseline	0.4932	0.2197	0	0
	completa	0.2098	0.0345	0.1450	0.0055
16	baseline	0.1513	0.0621	0	0
	completa	0.0625	0.0129	0.0363	0.0055
17	baseline	0.1536	0.0661	0	0
	completa	0.0658	0.0174	0.0318	0.0045

10.12.2. Qualidade da relaxação e efeito dos cortes

Os resultados da Tabela 20 mostram que a configuração completa fortalece sistematicamente a relaxação linear na raiz. Em praticamente todas as instâncias, o valor *root_lp_after_cuts* aproxima-se mais do ótimo inteiro do que o valor da relaxação original do *baseline*, reduzindo o *gap* inicial.

Em alguns casos, esse efeito é particularmente expressivo. Nas instâncias 02 e 04, os cortes foram suficientes para fechar totalmente o *gap* na raiz,

levando o *gap_root_after_cuts* a zero. Nas instâncias 05, 07, 08, 10, 12 e 17, também se observa uma redução significativa do *gap*, ainda que não total. Em 10, por exemplo, o *gap* cai de 0.0347 para 0.0015. Em 01, a redução é de 0.0556 para 0.0296. Em 12, de 0.0292 para 0.0098.

Esse comportamento indica que os planos de corte adicionados (MIG, MIR, *Lifting and Extending* e *Cover Cut*) conseguem reforçar a formulação logo no início da busca, melhorando os *bounds* duais e, conseqüentemente, aumentando o potencial de poda por *bound*. A configuração completa não altera apenas o percurso da árvore, mas também modifica a qualidade da relaxação.

Cabe notar que a instância 06 já possuía *gap* de raiz nulo no *baseline*, de modo que não havia espaço para melhoria nesse critério. Ainda assim, o resultado permanece consistente entre as duas configurações.

10.12.3. Redução do esforço da árvore

A diferença mais marcante entre as duas configurações aparece no tamanho da árvore de busca, conforme a Tabela 21. Em quase todas as instâncias, a configuração completa reduz de forma drástica o número de nós explorados, a profundidade máxima e a quantidade máxima de nós abertos.

Essa redução é especialmente pronunciada nas instâncias maiores. Na instância 15, o número de nós explorados cai de 2883 para 103, uma redução extremamente significativa, acompanhada pela queda da profundidade máxima de 16 para 8 e do número máximo de nós abertos de 888 para apenas 6. Na instância 16, a árvore diminui de 973 para 21 nós, com redução de profundidade de 11 para 5 e de nós abertos de 431 para 4. Na instância 17, a redução é de 807 para 35 nós. Nas instâncias 13 e 14, o número de nós explorados também cai fortemente, de 1191 para 253 e de 1115 para 139, respectivamente.

Esses resultados sugerem um efeito combinado de vários mecanismos. O *presolve* reduz o modelo ou fortalece limites antes mesmo do início da busca; os cortes melhoram os *bounds* duais na raiz e nos nós subsequentes; as heurísticas primais encontram incumbentes mais cedo; e a política de busca em profundidade, aliada à regra **PseudoCostRule**, tende a direcionar o solver com mais eficiência para regiões promissoras da árvore. O efeito líquido é uma poda mais agressiva e uma exploração muito menos dispersa do espaço de busca.

Há, porém, uma exceção relevante, a instância 08 apresenta aumento de nós explorados, de 47 no *baseline* para 65 na configuração completa, além

de incremento na profundidade máxima de 5 para 7. Esse comportamento reforça que a adição de mecanismos mais sofisticados não garante redução monotônica da árvore em todos os casos individuais. Dependendo da interação entre política de busca, heurísticas e cortes, o solver pode seguir um percurso alternativo que, embora potencialmente melhor informado, ainda assim resulta em uma árvore maior para uma instância específica. Ainda assim, no conjunto dos experimentos, o saldo é favorável à configuração completa.

10.12.4. Impacto nos tempos de execução

A Tabela 22 mostra que a redução drástica da árvore não implica necessariamente redução uniforme do tempo total. Esse é um resultado esperado, pois a configuração completa incorpora componentes adicionais com custo próprio: *presolve*, geração de cortes e execução recorrente de heurísticas primais.

Nas instâncias pequenas e médias, o custo adicional desses mecanismos frequentemente supera a economia obtida pela redução da árvore. Isso é visível, por exemplo, nas instâncias 01, 02, 05, 07, 08, 09, 10 e 12, em que o tempo total da configuração completa é superior ao do *baseline*. Em 08, por exemplo, o tempo total cresce de 0.0054 para 0.0739 s, com parcela dominante do tempo atribuída às heurísticas (0.0515 s). Em 07, o tempo aumenta de 0.0076 para 0.0353 s, novamente com forte participação do componente heurístico.

Por outro lado, nas instâncias mais difíceis, a redução da árvore passa a compensar esse custo adicional. Isso ocorre de forma clara nas instâncias 15, 16 e 17. Na instância 15, o tempo total cai de 0.4932 para 0.2098 s; na 16, de 0.1513 para 0.0625 s; e na 17, de 0.1536 para 0.0658 s. Nessas instâncias, a diminuição expressiva do número de nós explorados e do tempo total gasto em resolução de LPs supera o custo adicional com heurísticas e cortes.

O caso da instância 13 é ilustrativo, isto é, embora a árvore seja muito menor na configuração completa, o tempo total ainda aumenta de 0.1862 para 0.3093 s, principalmente devido ao custo das heurísticas (0.2273 s). Isso sugere que, para algumas instâncias, o agendamento de heurísticas pode estar excessivamente agressivo. Assim, os resultados indicam que a configuração completa melhora o comportamento da busca, mas seu custo computacional adicional nem sempre é compensado no tempo final, sobretudo quando a instância não é suficientemente difícil para justificar esse investimento.

10.12.5. Síntese da comparação

A configuração *baseline* fornece uma referência importante por isolar o comportamento essencial do *Branch-and-Bound*, sem mecanismos auxiliares. Já a configuração completa fortalece de maneira significativa a relaxação, reduz fortemente o tamanho da árvore e encontra incumbentes mais cedo e com maior frequência.

O principal ganho da configuração completa está, portanto, na qualidade do processo de busca: menos nós, menor profundidade, menos nós abertos e *bounds* mais fortes. Em contrapartida, esses ganhos estruturais vêm acompanhados de custo computacional adicional, sobretudo devido às heurísticas e aos cortes. Como consequência, a configuração completa não é sempre mais rápida em tempo total nas instâncias fáceis, mas tende a ser superior nas instâncias mais difíceis, nas quais a redução da árvore se torna decisiva.

Em síntese, os experimentos mostram que a combinação de *presolve*, cortes, heurísticas primais, *Depth First* e *PseudoCostRule* produz um solver substancialmente mais robusto do que o *baseline*. O ganho mais consistente não está apenas na redução do tempo em todas as instâncias, mas principalmente na redução do esforço combinatório necessário para provar otimalidade.

10.13. Teste com formulação SOS2

Para avaliar o suporte a restrições do tipo SOS2, foi construída uma instância de linearização *piecewise linear* da função $f(x) = x^2$, com *breakpoints* nos pontos $(0, 0)$, $(1, 1)$, $(2, 4)$, $(3, 9)$ e $(4, 16)$. O modelo considera as variáveis contínuas x e y , onde y representa a aproximação linear por partes de $f(x)$, e impõe a restrição adicional

$$x \geq 2.3.$$

O objetivo é minimizar y . A estrutura SOS2 foi modelada por meio de uma reformulação MILP equivalente com variáveis contínuas de convexidade λ_i e variáveis binárias de segmento z_j . Assim, o modelo final permanece linear e pode ser resolvido diretamente pelo solver *Branch-and-Bound* implementado. Ressalta-se que a configuração do solver aqui utilizado é o referente ao *baseline* cru, isto é, sem cortes, *presolve*, heurísticas e demais componentes. Entretanto, utilizou-se a estratégia de *Depth-First Search* ao invés de *Breadth-First Search*.

10.13.1. Estrutura da instância gerada

A instância gerada possui 11 variáveis e 10 restrições. Dentre as variáveis, 4 são binárias e 7 são contínuas. As 4 variáveis binárias correspondem à seleção de segmentos da formulação SOS2, enquanto as variáveis contínuas são compostas pelas variáveis originais x e y e pelas cinco variáveis λ_i .

10.13.2. Resultado

O solver encontrou solução ótima com valor objetivo 5.5000. Esse valor coincide com o valor esperado da aproximação *piecewise linear* no ponto $x = 2.3$, localizado no segmento entre os *breakpoints* 2 e 3. De fato, a interpolação linear nesse trecho produz

$$y = 0.7 \cdot 4 + 0.3 \cdot 9 = 5.5.$$

Além disso, o valor da relaxação linear na raiz também foi 5.5000, implicando *gap* de raiz nulo. Isso mostra que, para esta instância, a relaxação LP já fornece o valor ótimo do problema inteiro. Ainda assim, a solução da raiz não era inteira, pois continha variáveis binárias fracionárias associadas à escolha do segmento. Portanto, embora o *bound* da raiz já fosse ótimo, o solver ainda precisou explorar a árvore de *Branch-and-Bound* para obter uma solução factível inteira.

Foram explorados 5 nós no total, dos quais 2 foram podados. A profundidade máxima observada foi 2, e o número máximo de nós abertos simultaneamente foi 2, caracterizando uma árvore pequena.

O tempo total foi de 0.0048 s, dos quais 0.0040 s foram gastos na resolução das relaxações LP. Não houve tempo gasto com heurísticas ou planos de corte, pois esses mecanismos não foram utilizados neste experimento.

O experimento confirma que o solver implementado é capaz de resolver problemas com linearização *piecewise linear* baseada em SOS2 por meio de uma reformulação MILP equivalente.

Table 23: Resultado final e qualidade da relaxação - *SOS2*.

instance	status	obj_value	root_lp	root_lp_after_cuts	gap_root	gap_root_after_cuts
sos2_quad	optimal	5.5000	5.5000	—	0.0000	—

Table 24: Tamanho e esforço da árvore Branch-and-Bound - *SOS2*.

instance	nodes_explored	nodes_pruned	pruned_by_infeasibility	pruned_by_bound	max_depth	max_open_nodes
sos2_quad	5	2	1	1	2	2

Table 25: Tempos de execução por componente - *SOS2*.

instance	runtime_total	runtime_lp	runtime_heuristics	runtime_cuts
sos2_quad	0.0048	0.0040	0	0

11. Discussão

Os experimentos conduzidos permitem extrair conclusões consistentes sobre o impacto individual e combinado de cada componente do solver implementado.

Presolve. O pré-processamento mostrou custo computacional baixo e efeito heterogêneo. Em instâncias pequenas, as simplificações obtidas foram modestas e não alteraram significativamente a árvore. Já nas instâncias maiores, especialmente a 16, o presolve produziu reduções expressivas no número de nós explorados e no tempo total, indicando que o fortalecimento da formulação antes do início da busca pode ser decisivo quando o modelo admite simplificações estruturais relevantes.

Estratégia de exploração. A estratégia *breadth-first* mostrou-se a menos eficiente no conjunto testado, mantendo fronteiras muito largas e explorando mais nós nas instâncias maiores. A estratégia *depth-first* apresentou o comportamento mais robusto, com reduções frequentes no tamanho da árvore, menor uso de memória implícita e melhores tempos nas instâncias difíceis. A estratégia *best-first* foi competitiva em alguns casos, mas com desempenho menos uniforme, sugerindo maior dependência da estrutura da instância.

Regra de branching. Ambas as regras avançadas superaram a *baseline* nas instâncias maiores. O *strong branching* produziu as menores árvores, mas com custo por nó mais alto, o que nem sempre se converteu em menor tempo total. O *pseudocost branching* apresentou o melhor equilíbrio entre qualidade das decisões e custo computacional, sendo a alternativa mais vantajosa quando o critério de avaliação é o tempo total de execução.

Cortes válidos. Os planos de corte implementados (GMI, MIR, *Lifting* & *Extending* e *Cover Cut*) fortaleceram sistematicamente a relaxação linear na raiz, reduzindo o *gap* inicial e, em alguns casos, fechando-o completamente. Isso se refletiu em árvores menores e tempos menores nas instâncias mais difíceis. Contudo, em instâncias pequenas, o custo de separação superou o benefício obtido, tornando os cortes desfavoráveis nesses casos. O resultado indica que a geração de cortes é mais vantajosa quando a instância

é suficientemente difícil para que o fortalecimento da relaxação compense o *overhead*.

Heurísticas primais. As heurísticas foram eficazes para encontrar incumbentes mais cedo, intensificando a poda por *bound* e reduzindo o esforço estrutural da árvore. No entanto, seu custo de execução foi alto em todas as instâncias testadas, dominando o tempo total e superando com frequência a economia obtida na resolução de relaxações lineares. Isso sugere que a frequência de ativação das heurísticas deve ser calibrada com cuidado, especialmente em instâncias de porte pequeno a médio.

Configuração completa. A combinação de todos os mecanismos produziu reduções drásticas no tamanho da árvore, com ganhos de tempo expressivos nas instâncias mais difíceis. Nas instâncias menores, o custo adicional dos componentes superou os benefícios, resultando em tempos totais maiores do que o *baseline*. Esse comportamento é esperado e reflete o fato de que mecanismos sofisticados são mais vantajosos quando o problema é suficientemente difícil para justificar o investimento.

SOS2. O experimento com linearização *piecewise linear* confirmou que a reformulação MILP equivalente baseada em variáveis binárias de segmento é compatível com o solver implementado, sem necessidade de qualquer extensão. O resultado obtido foi correto e a árvore gerada foi pequena, validando a abordagem.

12. Conclusão

Este trabalho apresentou a implementação de um solver *Branch-and-Cut* para problemas de programação inteira mista, com arquitetura modular e plugável. O Gurobi foi utilizado exclusivamente para a resolução das relaxações LP em cada nó, enquanto todos os mecanismos de busca, fortalecimento e heurísticas foram implementados diretamente.

Os experimentos demonstraram que cada componente contribui de forma distinta para o desempenho do solver. O presolve é barato e pode ser muito eficaz quando a formulação admite simplificações. A estratégia de exploração *depth-first* mostrou-se a mais robusta no conjunto testado. O *pseudocost branching* ofereceu o melhor equilíbrio entre qualidade e custo. Os cortes válidos fortaleceram a relaxação de forma consistente, sendo mais vantajosos nas instâncias difíceis. As heurísticas primais reduziram o esforço combinatorio da busca, mas seu custo de execução exige calibração cuidadosa da frequência de ativação. Por fim, a configuração completa demonstrou que a

combinação integrada desses mecanismos produz um solver substancialmente mais robusto do que a versão mínima, com ganhos expressivos nas instâncias de maior porte.

Como trabalhos futuros, destacam-se: a implementação de um critério adaptativo de ativação de heurísticas baseado no progresso da busca; a extensão dos cortes para geração em nós internos da árvore, e não apenas na raiz; a incorporação de estratégias de seleção de nós híbridas, combinando profundidade e qualidade do *bound*; e a avaliação do solver em instâncias de maior porte provenientes de benchmarks padrão como o MIPLIB.